

RAPPORT D'ACTIVITE

Analyse de données – Parcours débutants

```
[+] Creating 1/1
✓ Network seance-01_default Created
0.0s
Bienvenue dans le cours d'analyse de données en géographie !
A
0 1
1 2
2 3

tariqcolombero@MacBook-Pro seance-01 %
```

Image d'illustration issue du résultat du code de la première séance.

Par Tariq COLOMBERO,

Étudiant en première année du Master GAED - SCT
Faculté des Lettres, Sorbonne Université, Paris IV

TABLE DES MATIERES

TABLE DES MATIERES	1
SEANCE 01.....	3
I. Objectifs.	3
II. Installation de Python via Docker.	3
III. Problème rencontré : installation de Fiona.	4
IV. Résolution du problème rencontré : installation de Fiona.	4
V. Vérification et conclusion.	6
SEANCE 02.....	7
I. Objectifs.	7
II. Questions de cours.	7
III. Mise en œuvre avec Python.....	12
1. Initialisation (étape 1 à 4).....	12
2. Manipulations (étape 5 à 7).....	12
3. Natures, noms et sélections des colonnes (étape 7 à 9).....	14
4. Total des effectifs (étape 10).	16
5. Matplotlib (étape 11 à 13).....	18
6. Étape Bonus.	23
SEANCE 03.....	25
I. Objectifs.	25
II. Questions de cours.	25
III. Mise en œuvre avec Python.....	28
1. Initialisation (étape 1 à 4).....	28
2. Manipulations premières (étape 5 à 6).	28
3. Manipulations secondaires (étape 7 à 8).....	29
4. Manipulations finales (étape 9 à 10).	31
5. Étape Bonus.	32
SEANCE 04.....	33
I. Objectifs.	33
II. Questions de cours.	33
III. Mise en œuvre avec Python.....	35
1. Mise en œuvre.	35
2. Les distributions statistiques de variables discrètes (étape 1 et 2).	35
3. Les distributions statistiques de variables continues (étape 1 et 2).	37
4. Visualisation avec Matplotlib (étape 1 et 2).	39
SEANCE 05.....	42
I. Objectifs.	42

II. Questions de cours.	42
III. Mise en œuvre avec Python.....	45
1. Théorie de l'échantillonnage (Étape 1).	45
2. Théorie de l'estimation (Étape 2).	47
3. Théorie de la décision (Étape 3).....	49
4. Bonus (Étape 4).	51
<i>SEANCE 06</i>.....	52
I. Objectifs.	52
II. Questions de cours.	52
III. Mise en œuvre avec Python.....	55
1. Fichier Island (Étape 1 à 7).	55
2. Fichier Le Monde (Étape 8 à 14).	58
3. Bonus.	60

SEANCE 01

I. Objectifs.

L'objectif de cette première séance était de mettre en place un environnement Python isolé à l'aide de Docker, tel que défini sur le feuille de route. L'environnement devait permettre :

- d'installer automatiquement Python (version 3.6).
- de charger les bibliothèques nécessaires à l'analyse de données (via requirements.txt).
- d'exécuter des scripts Python directement depuis le terminal à l'intérieur du conteneur Docker.

II. Installation de Python via Docker.

Après avoir installé Docker sur mon ordinateur personnel, et m'être connecté sur mon compte personnel, j'ai téléchargé les fichiers nécessaire à l'installation de Python.

Pour faire suite, il fallait sur le terminal de commande de MacOS se situer par rapport au dossier de la séance 01 qui contenait les fichiers suivants :

- docker-compose.yml
- requirements.txt
- Dockerfile.txt
- **src/** main.py

Pour se positionner sur MacOS dans le dossier de la séance 01, j'ai dû taper cette commande (puis faire la commande « `ls` » pour m'afficher les fichiers du dossier d'emplacement) :

```
tariqcolombero@MacBook-Pro ~ % cd "/Users/tariqcolombero/Desktop/Analyse de données/Colombero-2025-2026-Analyse-de-donnees/seance-01"
```

À partir de là, j'ai fait ma première tentative d'exécution du code en tapant la commande suivante :

```
tariqcolombero@MacBook-Pro seance-01 % docker-compose up -d --build
```

III. Problème rencontré : installation de Fiona.

Cette étape a conduit à une erreur récurrente au moment de l'installation de certaines bibliothèques Python. Plus précisément, l'échec concernait Fiona, une bibliothèque de traitement de données géospatiales.

L'analyse des messages d'erreur montrait une absence de fichiers critiques comme `cpl_conv.h`, et le message :

```
fatal error: cpl_conv.h: No such file or directory  
error: command 'gcc' failed with exit status 1
```

L'échec de l'installation ne provenait pas directement de Python, mais de la bibliothèque Fiona. Cette dernière n'est en effet pas une librairie purement Python : elle repose sur GDAL, un ensemble d'outils géospatiaux développés en C et en C++.

Lors de la compilation, Fiona avait donc besoin d'accéder à certains fichiers propres à GDAL, tels que les en-têtes `cpl_conv.h` ou encore l'outil de configuration `gdal-config`. Or, l'image de base utilisée dans le conteneur (python: 3.6) ne contenait pas ces éléments.

L'absence de ces dépendances système a conduit à l'échec de l'installation, puisque le compilateur ne parvenait pas à trouver les fichiers nécessaires pour construire Fiona correctement.

IV. Résolution du problème rencontré : installation de Fiona.

La ligne originale du fichier Dockerfile était la suivante :

```
RUN apt-get update && apt-get clean; rm -rf /var/lib/apt/lists/* /tmp/*  
/var/tmp/* /usr/share/doc/*
```

Cette commande ne faisait que mettre à jour et nettoyer le système, sans installer les dépendances nécessaires. Pour corriger le problème, je l'ai remplacée par :

```
RUN apt-get update && apt-get install -y \  
gdal \  
libgdal-dev \  
libproj-dev \  
libgeos-dev \  
build-essential \  
&& rm -rf /var/lib/apt/lists/*
```

Le rôle de chaque paquet est le suivant :

- gdal et libgdal-dev : fournissent les exécutables et les fichiers de développement de la librairie GDAL (Geospatial Data Abstraction Library), indispensable à Fiona pour manipuler des formats de données géospatiales (shapefiles, GeoJSON, raster, etc.). Ils contiennent notamment les fichiers manquants signalés par l'erreur, comme cpl_conv.h.
- libproj-dev et libgeos-dev : ajoutent respectivement le support des systèmes de projection cartographique (PROJ) et des opérations géométriques avancées (GEOS), qui sont souvent utilisés par les bibliothèques géospatiales Python (Fiona, GeoPandas, Shapely).
- build-essential : installe les outils de compilation nécessaires (gcc, g++, make, etc.) afin que Python puisse compiler les modules natifs des bibliothèques comme Fiona au moment de l'installation.

Grâce à l'ajout de ces paquets, Fiona a pu être compilée et installée correctement, ce qui a également permis l'installation sans erreur de bibliothèques dépendantes comme GeoPandas.

Après cette correction, j'ai relancé la construction complète de l'image avec :

```
tariqcolombero@MacBook-Pro seance-01 % docker-compose build --no-cache
```

L'installation a cette fois-ci abouti correctement. J'ai également, en parallèle, installé GDAL via Brew afin de régler le problème. GDAL n'est pas présent sur Mac Os par défaut.

V. Vérification et conclusion.

Plutôt que de tester un simple import Python, la vérification s'est faite en exécutant directement le fichier *main.py* situé dans le dossier **src/**, car ce fichier constitue le point d'entrée du projet. La commande utilisée fut :

```
tariqcolombero@MacBook-Pro seance-01 % docker-compose run python
```

Ce qui a produit le résultat suivant dans le terminal :

```
[+] Creating 1/1
✓ Network seance-01_default Created
0.0s
Bienvenue dans le cours d'analyse de données en géographie !
  A
0  1
1  2
2  3
tariqcolombero@MacBook-Pro seance-01 %
```

Ce résultat confirme que :

- le conteneur Docker fonctionne correctement.
- Python et les dépendances sont installés.
- le fichier *src/main.py* s'exécute bien comme prévu.

Pour clore le conteneur Docker, il fallait exécuter la commande suivante :

```
tariqcolombero@MacBook-Pro seance-01 % docker-compose down

[+] Running 1/1
✓ Network seance-01_default Removed
0.2s
tariqcolombero@MacBook-Pro seance-01 %
```

SEANCE 02

I. Objectifs.

- Manipuler un fichier C.S.V.
- Faire des sorties graphiques.
- Utiliser les bibliothèques Pandas (données) et Matplotlib (graphiques). N.B. pd et plt sont des alias qui remplacent respectivement pandas et matplotlib.pyplot.
- Calculer des effectifs.
- Calculer des fréquences.
- Faire des graphiques (diagrammes en bâton et circulaires, et histogrammes).

II. Questions de cours.

1. Quel est le positionnement de la géographie par rapport aux statistiques?

La géographie entretient une relation complexe et ambivalente avec la statistique. Historiquement, elle a souvent négligé les outils mathématiques, se considérant davantage comme une science humaine interprétative. Pourtant, la géographie produit des données massives et variées (socio-économiques, environnementales, spatiales) dont la compréhension nécessite l'usage d'outils statistiques.

Ainsi, la statistique permet au géographe de quantifier, comparer et modéliser les phénomènes spatiaux. Elle transforme la géographie en une science des échelles, capable de dégager des tendances globales à partir de faits locaux.

2. Le hasard existe-t-il en géographie?

Le hasard est avant tout une question philosophique. Deux positions existent : Déterministe (Laplace) : le hasard n'existe pas, tout phénomène a une cause identifiable. Et Probabiliste : le hasard traduit une ignorance des causes, mais il reste explicable. En géographie, il est impossible de prévoir chaque comportement individuel, mais possible

de dégager des tendances collectives probables (ex. migrations, urbanisation).

Le hasard devient donc mesurable et modélisable : les lois de probabilité (normale, Pareto, etc.) permettent de transformer l'incertitude en connaissance.

3. Quels sont les types d'information géographique .

Deux grands types :

Les informations attributaires : elles décrivent les caractéristiques d'un territoire (population, revenus, précipitations...). Les informations géométriques : elles concernent la forme et la structure des objets géographiques (polygones, lignes, points).

Ces deux dimensions forment la base d'un Système d'Information Géographique (SIG) : les attributs décrivent quoi et où, la géométrie décrit comment c'est organisé.

4. Quels sont les besoins de la géographie au niveau de l'analyse de données?

La géographie doit :

Collecter, structurer et documenter les données via des nomenclatures et métadonnées fiables. Analyser les structures internes des données à l'aide d'outils statistiques (corrélations, régressions, classifications, analyses factorielles). Visualiser les résultats sous forme de cartes, diagrammes ou modèles spatiaux.

L'objectif est de passer de la description à la compréhension, en reliant les faits observés à leurs logiques spatiales.

5. Quelles sont les différences entre la statistique descriptive et la statistique explicative?

La statistique descriptive et la statistique explicative constituent deux étapes complémentaires dans l'analyse des données géographiques.

La statistique descriptive vise avant tout à résumer, ordonner et représenter les données observées. Elle permet de mettre en évidence les grandes tendances d'un phénomène sans chercher à en identifier les

causes. Ses outils principaux sont les moyennes, médianes, écarts types, diagrammes, histogrammes ou encore les cartes thématiques.

En géographie, elle sert par exemple à décrire la population d'un territoire, à mesurer les densités, ou à cartographier la répartition des activités. En revanche, la statistique explicative (ou inférentielle) cherche à comprendre, modéliser et prévoir les phénomènes observés. Elle ne se limite pas à décrire : elle établit des relations de causalité entre variables — par exemple entre la répartition de la population et l'accès à l'emploi. Ses méthodes incluent les analyses de corrélation, les régressions simples ou multiples, les analyses factorielles ou les tests statistiques. En géographie, elle permet par exemple d'expliquer les causes de l'exode rural ou d'anticiper les dynamiques urbaines à partir de modèles probabilistes.

6. Quelles sont les types de visualisation de données en géographie? Comment choisir celles-ci?

- Diagrammes en barres / en secteurs : variables qualitatives (catégorielles).
- Histogrammes / boîtes à moustache : variables quantitatives continues.
- Cartes thématiques et choroplèthes : répartition spatiale.
- Nuages de points / cartes de chaleur : corrélations et densités.
- Graphes factoriels (ACP, AFC) : réduction de dimensions.

Le choix dépend du type de variable et du but de l'analyse (comparer, classer, représenter, corrélérer).

7. Quelles sont les méthodes d'analyse de données possibles?

Trois grandes familles :

1. Méthodes descriptives : ACP, AFC, ACM, CAH (résumer et visualiser les données).
2. Méthodes explicatives : régression simple/multiple, régression logistique, analyse discriminante (étudier les relations causales).
3. Méthodes de prévision : séries chronologiques, modèles temporels (prévoir les évolutions futures).

Ces méthodes permettent de passer d'un tableau de données à une structure spatiale intelligible.

8. Comment définiriez-vous : (a) population statistique? (b) individu statistique ? (c) caractères statistiques? (d) modalités statistiques? Quels sont les types de caractères? Existe-t-il une hiérarchie entre eux?

En géographie comme dans toute science de données, il est essentiel de maîtriser le vocabulaire statistique pour pouvoir structurer correctement une analyse. Voici les notions clés :

Terme	Définition	Exemple géographique
(a) Population statistique	Ensemble complet des unités observées, sur lesquelles porte l'étude.	Toutes les communes françaises, toutes les régions d'Europe, ou encore l'ensemble des bassins versants d'un pays.
(b) Individu statistique	Élément particulier appartenant à la population statistique.	Une commune donnée (ex. : Nîmes), une région (ex. : Île-de-France) ou un bassin versant spécifique.
(c) Caractère statistique	Propriété ou variable étudiée pour chaque individu.	Le taux de chômage, la population totale, le revenu moyen, la température moyenne annuelle.
(d) Modalité statistique	Valeur prise par un caractère pour un individu donné.	12 % de chômage, 250 000 habitants, 19 °C de moyenne annuelle.

9. Comment mesurer une amplitude et une densité?

- **Amplitude (A)** : étendue d'une classe statistique

$$A = b - a$$

avec a = borne inférieure, b = borne supérieure.

- **Densité (d)** : rapport entre l'effectif et l'amplitude

$$d = \frac{n_i}{b - a}$$

Elle permet de comparer les effectifs entre classes d'amplitudes différentes.

10. À quoi servent les formules de Sturges et de Yule?

Elles servent à déterminer le nombre optimal de classes (k) dans un histogramme :

- **Sturges :**

$$k \approx 1 + 3,322 \times \log_{10}(n)$$

- **Yule :**

$$k \approx 2,5 \times n^{1/4}$$

Ces formules évitent un découpage trop fin (bruit) ou trop grossier (perte d'information).

11. Comment définir un effectif ? Comment calculer une fréquence et une fréquence cumulée? Qu'est-ce qu'une distribution statistique ?

- **Effectif (n_i)** : nombre d'occurrences d'une modalité.
- **Fréquence (f_i)** : part relative de cette modalité dans la population totale.

$$f_i = \frac{n_i}{n}$$

- **Fréquence cumulée** : somme progressive des fréquences jusqu'à une modalité donnée.

$$F_k = \sum_{i=1}^k f_i$$

- **Distribution statistique** : répartition des effectifs (ou fréquences) selon les modalités d'un caractère.

Elle permet d'identifier les tendances globales et de relier la statistique descriptive à la probabilité théorique.

III. Mise en œuvre avec Python.

1. Initialisation (étape 1 à 4).

Après avoir téléchargé les fichiers de la séance 02, et s'être situé dans le dossier de la séance avec la commande suivante :

```
tariqcolombero@MacBook-Pro seance-02 %  
cd "/Users/tariqcolombero/Desktop/Analyse de données/Colombero-2025-2026-  
Analyse-de-donnees/seance-02"
```

Nous pouvons ensuite ouvrir le fichier *main.py* et y trouver le code suivant :

```
#coding:utf8  
  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# Source des données : https://www.data.gouv.fr/datasets/election-  
presidentielle-des-10-et-24-avril-2022-resultats-definitifs-du-1er-tour/  
  
with open("./data/resultats-elections-presidentielles-2022-1er-  
tour.csv", "r") as fichier:  
    contenu = pd.read_csv(fichier)  
  
# Mettre dans un commentaire le numéro de la question  
# Question 1  
# ...
```

Le code ci-dessus montre comment lire le fichier CSV recensant les résultats des élections présidentielles du 1^{er} tour de 2022, téléchargé dans le dossier **data**.

2. Manipulations (étape 5 à 7).

La 5^{ème} étape à réaliser est d'afficher le contenu du tableau du fichier CSV sur le terminal avec le code suivant :

```
#5 Afficher le contenu du tableau  
print(contenu)
```

Ce qui nous donne le résultat suivant (non-complet) :

```

Code du département      Libellé du département ... Prénom.11
Voix.11
0          01          Ain ... Nicolas
8998.0
1          02          Aisne ... Nicolas
5790.0
2          03          Allier ... Nicolas
4216.0
3          04          Alpes-de-Haute-Provence ... Nicolas
2504.0
4          05          Hautes-Alpes ... Nicolas
2142.0
..          ...          ...          ...
...
102         ZP          Polynésie française ... Nicolas
1969.0
103         ZS          Saint-Pierre-et-Miquelon ... Nicolas
82.0
104         ZW          Wallis et Futuna ... Nicolas
244.0
105         ZX          Saint-Martin/Saint-Barthélemy ... Nicolas
339.0
106         ZZ          Français établis hors de France ... Nicolas
7074.0

[107 rows x 56 columns]
tariqcolombero@MacBook-Pro seance-02 %

```

La 6^{ème} étape à réaliser est de calculer le nombre de lignes et de colonnes grâce à la fonction *len* (...), ce qui nous donne comme code :

```

#6 Calculer le nombre de lignes et de colonnes
print("Nombre de lignes :", len(contenu))
print("Nombre de colonnes :", len(contenu.columns))

```

Et ce qui nous donne sur le terminal le résultat suivant :

```

Nombre de lignes : 107
Nombre de colonnes : 56

```

3. Natures, noms et sélections des colonnes (étape 7 à 9).

La 7^{ème} étape de l'exercice est plus complexe. Il faut réaliser une liste sur le type de chaque colonnes en utilisant les fonctions *int*, *float*, *str* et *bool*.

Pour ce faire, il faut d'abord afficher sur le terminal les types détectés par Panda.

```
#7 Liste sur le type de chaque colonne
print(contenu.dtypes)
```

Ce qui nous donne (à gauche le début, à droite la suite et fin) :

```
Nombre de lignes : 107
Nombre de colonnes : 56
Code du département      object
Libellé du département   object
Inscrits                  int64
Abstentions              float64
Votants                   float64
Blancs                    float64
Nuls                      float64
Exprimés                  float64
Sexe                      object
Nom                       object
Prénom                    object
Voix                      float64
Sexe.1                    object
Nom.1                     object
Prénom.1                  object
Voix.1                    float64
Sexe.2                    object
Nom.2                     object
Prénom.2                  object
Voix.2                    float64
Sexe.3                    object
Nom.3                     object
Prénom.3                  object
Voix.3                    float64
Sexe.4                    object
Nom.4                     object
Prénom.4                  object
Voix.4                    float64
Sexe.5                    object
Nom.5                     object
Prénom.5                  object
Voix.5                    float64
Sexe.6                    object
Nom.6                     object
Prénom.6                  object
Voix.6                    float64
Sexe.7                    object
Nom.7                     object
Prénom.7                  object
Voix.7                    float64
Sexe.8                    object
Nom.8                     object
Prénom.8                  object
Voix.8                    float64
Sexe.9                    object
Nom.9                     object
Prénom.9                  object
Voix.9                    float64
Sexe.10                   object
Nom.10                    object
Prénom.10                 object
Voix.10                   float64
Sexe.11                   object
Nom.11                    object
Prénom.11                 object
Voix.11                   float64
dtype: object
```

Afin de bien comprendre la structure du fichier CSV fourni, il convient de distinguer la nature de chaque variable en fonction de son type informatique.

- ***object*** : correspond à des chaînes de caractères (*str* en Python). Ce sont des variables qualitatives nominales, c'est-à-dire du texte sans ordre particulier.
→ Exemple : Code du département, Libellé du département, Nom, Prénom.
- ***int64*** : correspond à des entiers (*int* en Python). Ce sont des variables quantitatives discrètes, utilisées pour compter des individus ou des occurrences.
→ Exemple : Inscrits, Abstentions, Votants, Blancs, Nuls, Exprimés.
- ***float64*** : correspond à des nombres décimaux (*float* en Python). Ce sont des variables quantitatives continues, souvent utilisées pour des proportions, pourcentages ou résultats arrondis.
→ Exemple : Voix attribuées à un candidat.
- ***bool*** : correspond à une valeur binaire (Vrai/Faux). Ces variables sont rares dans ce type de fichier électoral, mais elles peuvent apparaître dans certains jeux de données. Ne figure pas ici.

La 8^{ème} étape du code est d'afficher sur le terminal le nom des colonnes avec la méthode Pandas head avec le code suivant :

```
#8 Liste sur le type de chaque colonne
print("Aperçu du tableau:")
print(contenu.head())
```

Ce qui nous donne le résultat suivant sur le terminal (capture d'écran):

```
Aperçu du tableau avec head() :
-bound method NDFrame.head of
Prénom.11 Voix.11
0 01 Ain 438109 97541.0 340568.0 5641.0 1903.0 333024.0 F ARTHAUD ... Valérie 17572.0 M POUTOU Philippe 2172.0 M DUPONT-AIGNAN Nicolas 8998.0
1 02 Aisne 373544 101089.0 272455.0 3767.0 2828.0 265860.0 F ARTHAUD ... Valérie 18920.0 M POUTOU Philippe 2118.0 M DUPONT-AIGNAN Nicolas 5790.0
2 03 Allier 249991 58497.0 191494.0 3749.0 1790.0 185955.0 F ARTHAUD ... Valérie 18319.0 M POUTOU Philippe 1583.0 M DUPONT-AIGNAN Nicolas 4216.0
3 04 Alpes-de-Haute-Provence 128075 29290.0 80785.0 1478.0 624.0 96683.0 F ARTHAUD ... Valérie 3334.0 M POUTOU Philippe 865.0 M DUPONT-AIGNAN Nicolas 2594.0
4 05 Hautes-Alpes 113519 25337.0 88162.0 1395.0 532.0 86235.0 F ARTHAUD ... Valérie 4511.0 M POUTOU Philippe 801.0 M DUPONT-AIGNAN Nicolas 2142.0
... ..
102 ZP Polynésie Française 205576 142121.0 63455.0 1490.0 1935.0 60830.0 F ARTHAUD ... Valérie 4725.0 M POUTOU Philippe 451.0 M DUPONT-AIGNAN Nicolas 1969.0
103 ZS Saint-Pierre-et-Miquelon 5045 2272.0 2773.0 47.0 25.0 2701.0 F ARTHAUD ... Valérie 51.0 M POUTOU Philippe 50.0 M DUPONT-AIGNAN Nicolas 82.0
104 ZW Wallis et Futuna 9528 4125.0 5483.0 27.0 17.0 5359.0 F ARTHAUD ... Valérie 1354.0 M POUTOU Philippe 48.0 M DUPONT-AIGNAN Nicolas 244.0
105 ZX Saint-Martin/Saint-Barthélemy 24414 15812.0 8602.0 150.0 84.0 8368.0 F ARTHAUD ... Valérie 354.0 M POUTOU Philippe 63.0 M DUPONT-AIGNAN Nicolas 339.0
106 ZZ Français établis hors de France 1435746 931455.0 504291.0 3193.0 2347.0 498951.0 F ARTHAUD ... Valérie 20950.0 M POUTOU Philippe 3145.0 M DUPONT-AIGNAN Nicolas 7074.0
```


La 9^{ème} étape consiste à sélectionner la colonne des Inscrits :

```
#9 Sélectionner la colonne "Inscrits"  
print("Nombre des inscrits par départements :")  
print(contenu.Inscrits)
```

Ce qui nous donne le résultat suivant :

```
Nombre des inscrits par départements :  
0      438109  
1      373544  
2      249991  
3      128075  
4      113519  
...  
102    205576  
103     5045  
104     9528  
105    24414  
106   1435746
```

4. Total des effectifs (étape 10).

L'étape 10 est plus difficile. Il s'agit de calculer la somme des effectifs pour chaque catégorie (colonne) avec la méthode Pandas sum. Dans un premier temps j'ai saisi le code suivant :

```
#10 Calculer les effectifs de chaque colonnes  
print("Effectifs de chaque colonne :")  
print(contenu.sum())
```

Ce qui m'a donné cela comme résultat :

```

Effectifs de chaque colonne :
Code du département 01020304050607080910111213141516171819202122...
Libellé du département AinAisneAllierAlpes-de-Haute-ProvenceHautes-Al...
Inscrits 48747876
Abstentions 1.28242e+07
Votants 3.59237e+07
Blancs 543609
Nuls 247151
Exprimés 3.51329e+07
Sexe FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF...
Nom ARTHAUDARTHAUDARTHAUDARTHAUDARTHAUDARTH...
Prénom NathalieNathalieNathalieNathalieNathalieNathalieNathal...
Voix 197094
Sexe.1 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.1 ROUSSELROUSSELROUSSELROUSSELROUSSELROUSSELROUSSELROUS...
Prénom.1 FabienFabienFabienFabienFabienFabienFabienFabienFabi...
Voix.1 802422
Sexe.2 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.2 MACRONMACRONMACRONMACRONMACRONMACRONMACRONMACRONMACR...
Prénom.2 EmmanuelEmmanuelEmmanuelEmmanuelEmmanuelEmmanuelEmmanu...
Voix.2 9.78306e+06
Sexe.3 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.3 LASSALLELASSALLELASSALLELASSALLELASSALLELASSALLELASSAL...
Prénom.3 JeanJeanJeanJeanJeanJeanJeanJeanJeanJeanJeanJeanJe...
Voix.3 1.10139e+06
Sexe.4 FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF...
Nom.4 LE PENLE PENLE PENLE PENLE PENLE PENLE PENLE P...
Prénom.4 MarineMarineMarineMarineMarineMarineMarineMarineMari...
Voix.4 8.13383e+06
Sexe.5 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.5 ZEMMOURZEMMOURZEMMOURZEMMOURZEMMOURZEMMOURZEMMOURZEMM...
Prénom.5 ÉricÉricÉricÉricÉricÉricÉricÉricÉricÉricÉricÉricÉricÉr...
Voix.5 2.48523e+06
Sexe.6 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.6 MÉLÉNCHONMÉLÉNCHONMÉLÉNCHONMÉLÉNCHONMÉLÉNCHONMÉLÉNCHONM...
Prénom.6 Jean-LucJean-LucJean-LucJean-LucJean-LucJean-LucJean-L...
Voix.6 7.71252e+06
Sexe.7 FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF...
Nom.7 HIDALGOHIDALGOHIDALGOHIDALGOHIDALGOHIDALGOHIDALGOHIDA...
Prénom.7 AnneAnneAnneAnneAnneAnneAnneAnneAnneAnneAnneAnneAn...
Voix.7 616478
Sexe.8 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.8 JADOTJADOTJADOTJADOTJADOTJADOTJADOTJADOTJADOTJ...
Prénom.8 YannickYannickYannickYannickYannickYannickYannickYann...
Voix.8 1.62785e+06
Sexe.9 FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF...
Nom.9 PÉCRESSEPÉCRESSEPÉCRESSEPÉCRESSEPÉCRESSEPÉCRESSEPÉCRES...
Prénom.9 ValérieValérieValérieValérieValérieValérieValérieValé...
Voix.9 1.679e+06
Sexe.10 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.10 POUTOUPOUTOUPOUTOUPOUTOUPOUTOUPOUTOUPOUTOUPOUTOUP...
Prénom.10 PhilippePhilippePhilippePhilippePhilippePhilippePhilip...
Voix.10 268904
Sexe.11 MAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Nom.11 DUPONT-AIGNANDUPONT-AIGNANDUPONT-AIGNANDUPONT-AIGNANDUPONT-...
Prénom.11 NicolasNicolasNicolasNicolasNicolasNicolasNicolasNico...
Voix.11 725176
dtype: object

```

Un résultat « bizarre », normal, car il manque la condition permettant de ne pas afficher les colonnes aux valeurs qualitatives :

```

#10 Calculer les effectifs de chaque colonnes
print("\nNombre des inscrits par département :")
print(contenu["Inscrits"])
print("\nSommes colonnes numériques :\n",
contenu.select_dtypes(include=["int64","float64"]).sum())

```

Ce qui nous donne le résultat suivant, harmonisé aux seules colonnes quantitatives :

```
Sommes des colonnes numériques :  
- Inscrits : 48747876  
- Abstentions : 12824169.0  
- Votants : 35923707.0  
- Blancs : 543609.0  
- Nuls : 247151.0  
- Exprimés : 35132947.0  
- Voix : 197094.0  
- Voix.1 : 802422.0  
- Voix.2 : 9783058.0  
- Voix.3 : 1101387.0  
- Voix.4 : 8133828.0  
- Voix.5 : 2485226.0  
- Voix.6 : 7712520.0  
- Voix.7 : 616478.0  
- Voix.8 : 1627853.0  
- Voix.9 : 1679001.0  
- Voix.10 : 268904.0  
- Voix.11 : 725176.0
```

5. Matplotlib (étape 11 à 13).

Il nous est demandé à l'étape 11 de faire des diagrammes en barres avec le nombre des inscrits et des votants pour chaque département. Pour ce faire, je me suis aidé du tutoriel en ligne de Matplotlib¹. Il faut dans un premier temps créer un dossier « Images » afin de faciliter le dépôt du graphique créé, avec **os** :

```
# Créer le dossier "images", s'il n'existe pas déjà.  
import os  
os.makedirs("images", exist_ok=True)
```

Par la suite, il faut, à l'aide d'une boucle, définir les colonnes « Inscrits » et « Votants » à tracer. Grace à une boucle, nous devons pour chaque colonne tracer le diagramme, mettre en forme et enregistrer les fichiers :

```
# 11. Diagrammes en barres : inscrits & votants  
os.makedirs("images", exist_ok=True)  
for c in ["Inscrits", "Votants"]:  
    plt.figure(figsize=(12,6))
```

¹ https://matplotlib.org/stable/plot_types/basic/bar.html#sphx-glr-plot-types-basic-bar-py

```
plt.bar(contenu["Libellé du département"], contenu[c],
color="cornflowerblue")
plt.xticks(rotation=90)
plt.title(f'Nombre de {c} par département')
plt.ylabel("Nombre d'électeurs")
plt.tight_layout()
plt.savefig(f'images/{c}.png', dpi=300)
plt.close()
print("→ Diagrammes 'Inscrits.png' et 'Votants.png' enregistrés")
```

J'ai précisé l'enregistrement du fichier en .png avec un dpi égal à 300, afin d'avoir une qualité de rendu meilleure, à la suite de quelques tentatives ratées, car le fichier rendu était trop « brouillon ». Ce qui nous donne le rendu suivant :

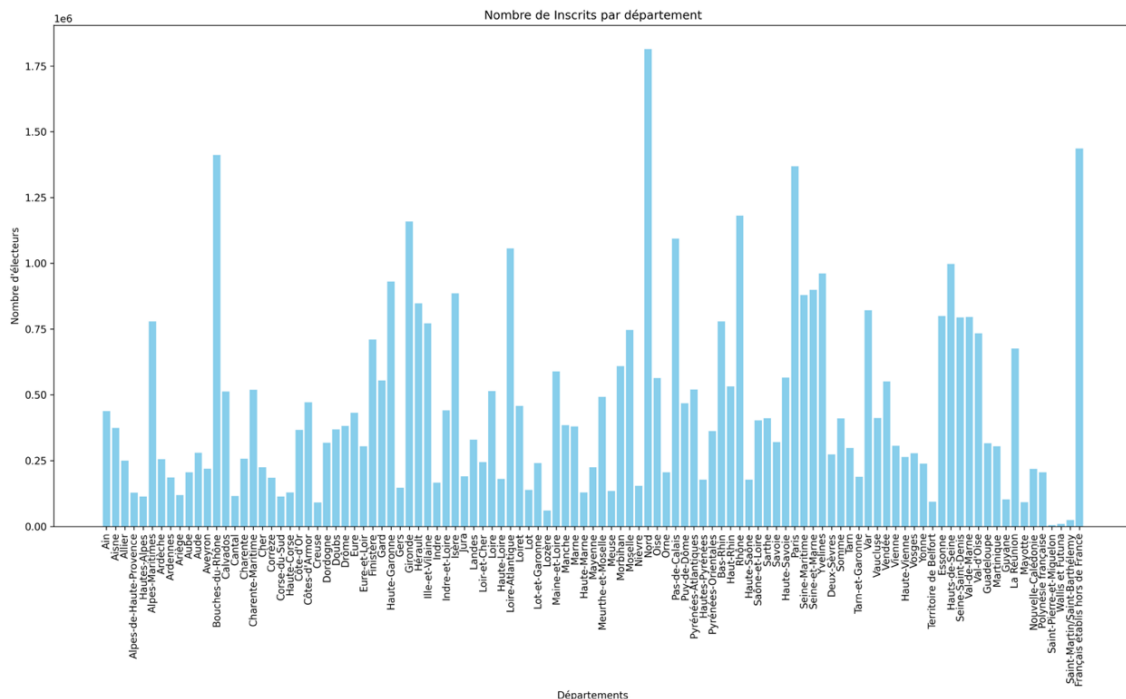


Fig. 1 : Nombre de Inscrits par département.

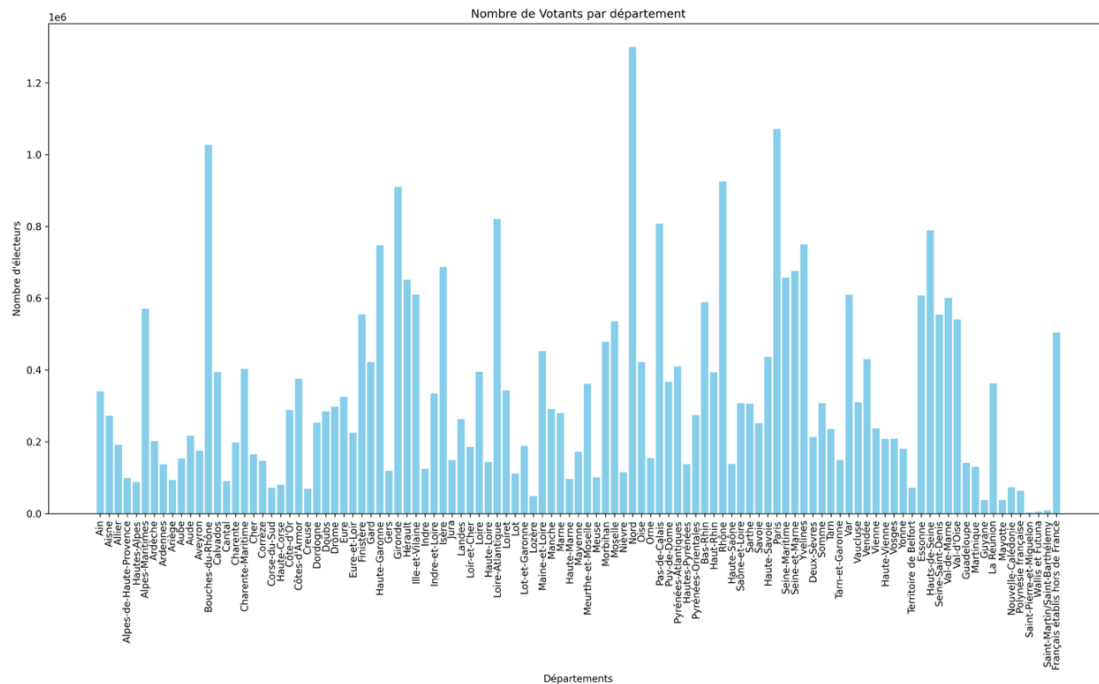


Fig. 2 : Nombre de Votants par département.

Or, il y a une erreur dans ma compréhension de la consigne. C'est pourquoi il a du tout refaire pour l'étape 11. Voici le code suivant retapé, et les figures que cela était censé faire apparaître :

```
# 11. Diagrammes en barres : inscrits & votants
import os
os.makedirs("Images", exist_ok=True)

departements = contenu["Libellé du département"].unique()

for dept in departements:
    data_dept = contenu[contenu["Libellé du département"] == dept]

    inscrits = data_dept["Inscrits"].values[0]
    votants = data_dept["Votants"].values[0]

    plt.figure(figsize=(5,4))
    plt.bar(["Inscrits", "Votants"], [inscrits, votants])
    plt.title(f'{dept} : Inscrits vs Votants')
    plt.ylabel("Nombre d'électeurs")
    plt.tight_layout()
    plt.savefig(f'Images/{dept}.png', dpi=300)
    plt.close()

print("> Diagrammes individuels par département générés dans le dossier Images/")
```

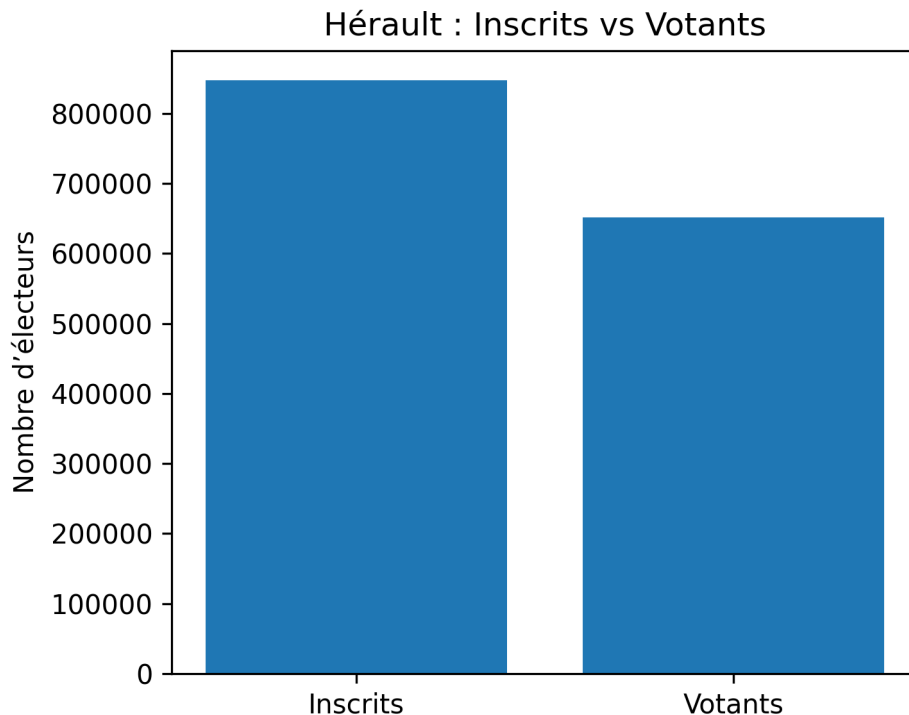


Fig. 2bis : Diagramme en barre du département de l'Hérault.

La 12^{ème} étape correspond au tracé de diagrammes circulaires avec les votes blancs, nuls, exprimés et abstention pour chaque département.

De la même manière que l'étape précédente, il faut créer un dossier, puis définir les colonnes à tracer et créer une boucle pour chaque département comportant la création de la figure, son titre, les données du département et l'enregistrement de l'image. Ce qui nous donne le code suivant :

```
# 12. Diagrammes circulaires : blancs, nuls, exprimés, abstentions
os.makedirs("images_pie", exist_ok=True)
cols_pie = ["Blancs", "Nuls", "Exprimés", "Abstentions"]
for _, row in contenu.iterrows():
    dep = re.sub(r"^\w\-", "-", row["Libellé du département"])
    plt.figure(figsize=(5,5))
    plt.pie([row[c] for c in cols_pie], labels=cols_pie, autopct="%1.1f%%",
startangle=90)
    plt.title(f"Répartition des votes - {row['Libellé du département']}")
    plt.tight_layout()
    plt.savefig(f"images_pie/{row['Code du département']}-{dep}.png",
dpi=300)
```

```
plt.close()
print("→ Diagrammes circulaires enregistrés dans /images_pie")
```

Ce qui nous donne le résultat suivant avec l'exemple de département du Gard (30) et de l'Hérault (34) :

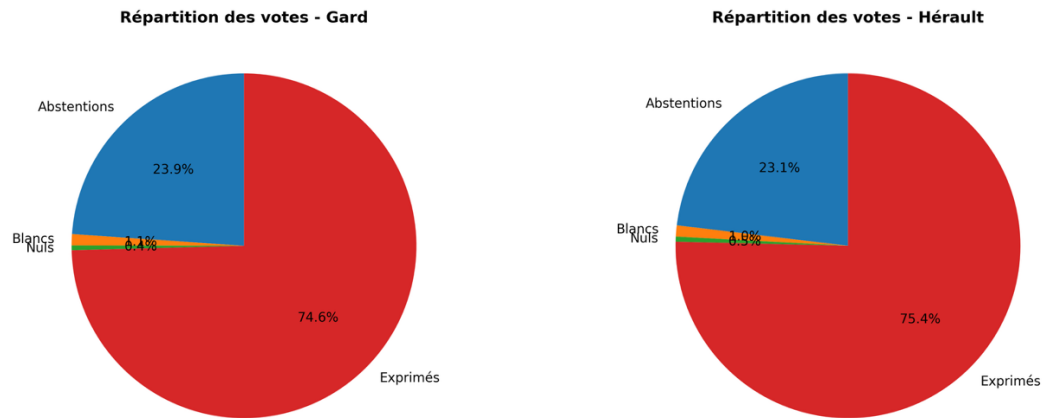


Fig. 3 : Répartition des votes pour le département du Gard (30) et de l'Hérault (34).

La 13^{ème} et dernière étape correspond à la création d'un histogramme de la colonne « Inscrits » de la même manière que les étapes précédentes :

```
# 13. Histogramme de la distribution des inscrits
os.makedirs("images_hist", exist_ok=True)
plt.figure(figsize=(10,6))
plt.hist(contenu["Inscrits"], bins=20, color="seagreen", edgecolor="black",
density=True)
plt.title("Distribution des inscrits")
plt.xlabel("Nombre d'inscrits")
plt.ylabel("Densité")
plt.tight_layout()
plt.savefig("images_hist/histogramme_inscrits.png", dpi=300)
plt.close()
print("→ Histogramme enregistré dans /images_hist")
```

Ce qui nous donne le résultat suivant :

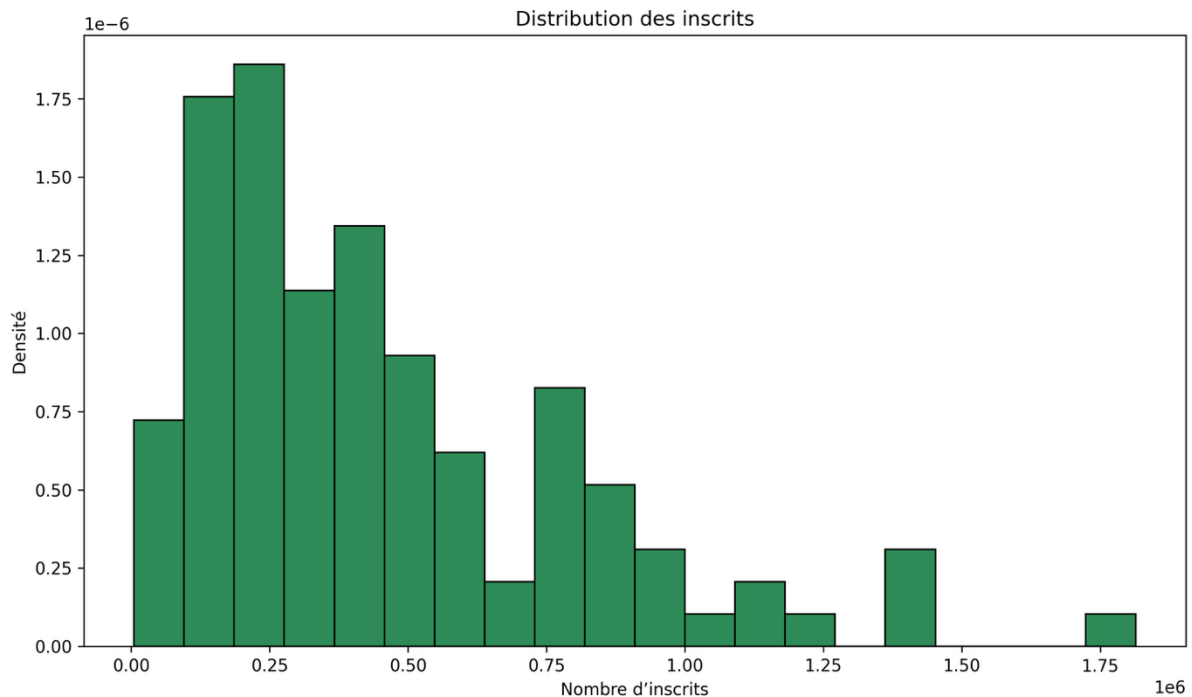


Fig. 4 : Histogramme des Inscrits.

6. Étape Bonus.

L'étape Bonus correspond à la réalisation de diagrammes circulaires pour chaque département et pour la France entière selon le nombre de voix par candidat.

Pour ce faire, on identifie premièrement les colonnes de voix, ensuite pour le diagramme circulaire par département on attribue la valeur des voix, on récupère les noms des candidats, puis on crée le diagramme avant de mettre en forme et d'enregistrer. On effectue la même chose pour la France entière, ce qui nous donne la commande suivante :

```
# 14. BONUS : diagrammes circulaires des voix par candidat
os.makedirs("images_voix", exist_ok=True)
vcols = [c for c in contenu.columns if c.startswith("Voix")]
def noms(r): return [r.get(f'Prénom.{i}', r['Prénom']) + " " +
r.get(f'Nom.{i}', r['Nom']) for i in range(len(vcols))]
for _, r in df.iterrows():
    dep = re.sub(r"^\w\-", "_", r["Libellé du département"])
    plt.pie(r[vcols], labels=noms(r), autopct='%1.1f%%', startangle=90)
    plt.title(r["Libellé du département"])
    plt.savefig(f"images_voix/{r['Code du département']}_{dep}.png",
dpi=300)
```



```
plt.close()
plt.pie(contenu[vcols].sum(), labels=noms(df.iloc[0]), autopct='%1.1f%%',
startangle=90)
plt.title("France entière")
plt.savefig("images_voix/France.png", dpi=300)
plt.close()
print("→ Diagrammes voix par candidat enregistrés (départements + France)")
```

Ce qui nous donne le résultat suivant pour le Gard (30) et l'Hérault (34) ainsi que la France entière :

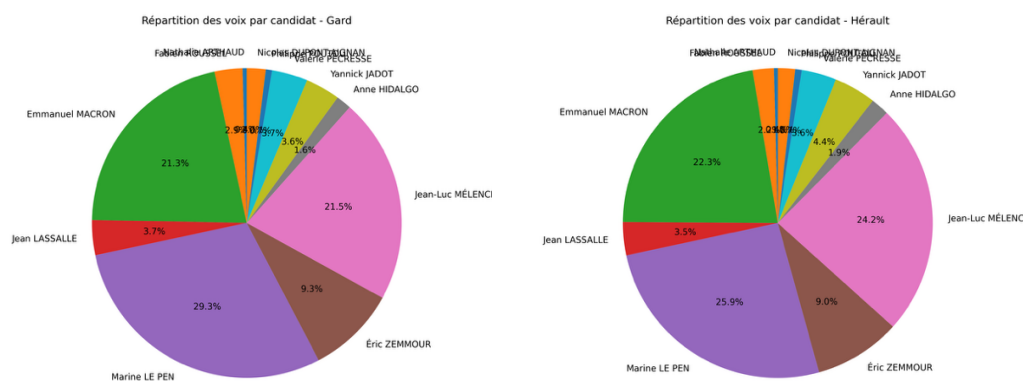


Fig. 5 : Répartition des voix pour chaque candidat pour le département du Gard (30) et de l'Hérault (34).

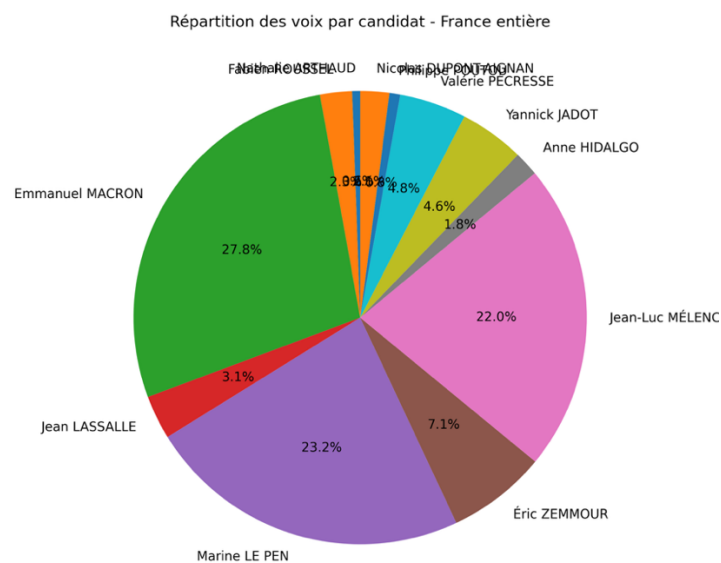


Fig. 6 : Répartition des voix pour chaque candidat en France.

SEANCE 03

I. Objectifs.

- Découvrir les méthodes de Pandas permettant de calculer les paramètres d'une série Statistique.
- Tracer une boîte de dispersion.

II. Questions de cours.

1. Quel caractère est le plus général : le caractère quantitatif ou le caractère qualitatif ? Justifier pourquoi.

Le caractère qualitatif est le plus général, car il englobe toutes les situations où l'on décrit une population par des modalités non numériques, comme la couleur, le type de sol, le sexe, la profession ou le régime politique. En revanche, le caractère quantitatif est un cas particulier du caractère qualitatif : il s'applique seulement lorsque les modalités peuvent être traduites en valeurs numériques mesurables (revenu, superficie, âge, altitude, etc.). Ainsi, tout caractère quantitatif est aussi un caractère qualitatif (car il permet de distinguer des modalités), mais l'inverse n'est pas vrai : un caractère qualitatif ne se mesure pas nécessairement.

2. Que sont les caractères quantitatifs discrets et caractères quantitatifs continus ? Pourquoi les distinguer ?

Les caractères quantitatifs discrets sont ceux dont les valeurs sont dénombrables, c'est-à-dire qui ne peuvent prendre qu'un nombre fini ou dénombrable de valeurs possibles : nombre d'enfants, nombre de logements, nombre de communes, etc. Les caractères quantitatifs continus, eux, peuvent prendre toute valeur dans un intervalle réel, comme la température, la superficie, le revenu ou la distance. On les distingue parce qu'ils n'impliquent pas les mêmes méthodes de traitement statistique :

- un caractère discret se traite par des comptages et fréquences ;

- un caractère continu nécessite des classes (ou intervalles) pour regrouper les valeurs, car elles sont infinies.

Cette distinction est donc essentielle pour choisir la bonne représentation graphique (histogramme ou diagramme en bâtons) et les bons paramètres de synthèse.

3. Paramètres de position. Pourquoi existe-t-il plusieurs types de moyenne ? Pourquoi calculer une médiane ? Quand est-il possible de calculer un mode ?

Il existe plusieurs types de moyenne (arithmétique, géométrique, harmonique, pondérée) parce qu'elles ne traduisent pas la même réalité statistique.

- La moyenne arithmétique s'applique à des valeurs homogènes et additive (par exemple, un revenu moyen).
- La moyenne géométrique est utilisée pour des évolutions relatives (croissance, indices).
- La moyenne harmonique s'emploie pour des vitesses ou des ratios.

La médiane représente la valeur centrale d'une série ordonnée : elle sépare la population en deux effectifs égaux. On la calcule lorsque la distribution est asymétrique ou contient des valeurs extrêmes : elle résume mieux la tendance centrale que la moyenne dans ce cas. Le mode correspond à la valeur la plus fréquente. On peut le calculer seulement lorsque la variable présente une ou plusieurs modalités qui se répètent. Il s'applique aussi bien à des caractères qualitatifs (la catégorie la plus fréquente) qu'à des caractères quantitatifs (la valeur la plus représentée dans un histogramme).

4. Paramètres de concentration. Quel est l'intérêt de la médiale et de l'indice de C. Gini ?

La médiale est une valeur qui partage la surface totale d'un histogramme en deux aires égales ; elle permet de mesurer la concentration d'une distribution, notamment lorsqu'on étudie la répartition d'une ressource (ex. : revenu, richesse). L'indice de Gini est un indicateur synthétique de l'inégalité de répartition. Il mesure la

distance entre la distribution réelle et une distribution parfaitement égalitaire. Il varie entre 0 (égalité parfaite) et 1 (inégalité totale). Plus il est proche de 1, plus la concentration est forte (par exemple, une minorité détient une grande part de la ressource). Ces deux indicateurs servent donc à évaluer la justice spatiale ou sociale d'un phénomène, enjeu central en géographie et en économie territoriale.

5. Paramètres de dispersion. Pourquoi calculer une variance à la place de l'écart à la moyenne ? Pourquoi la remplacer par l'écart type ? Pourquoi calculer l'étendue ? À quoi sert-il de créer un quantile ? Quel(s) est (sont) le(s) quantile(s) le(s) plus utilisé(s) ? Pourquoi construire une boîte de dispersion ? Comment l'interpréter ?

Les paramètres de dispersion mesurent l'hétérogénéité d'une distribution autour de sa moyenne. L'écart à la moyenne, qui additionne des valeurs positives et négatives, peut s'annuler ; c'est pourquoi on calcule plutôt la variance, en élevant ces écarts au carré pour éviter toute compensation. Toutefois, la variance s'exprime dans une unité au carré, peu intuitive : on la remplace donc par son écart-type, qui retrouve l'unité initiale (euros, km, habitants...). L'étendue (différence entre la valeur maximale et minimale) indique l'amplitude totale de variation ; elle est simple mais très sensible aux valeurs extrêmes. Les quantiles (quartiles, déciles, centiles) divisent la distribution en parts égales et permettent d'analyser la répartition d'un phénomène, par exemple en repérant les 10 % de communes les plus denses. Les plus utilisés sont les quartiles, notamment le deuxième (la médiane).

La boîte à moustaches ou boîte de dispersion représente visuellement ces quartiles : la boîte s'étend de Q_1 à Q_3 avec la médiane au centre, et les moustaches indiquent les valeurs extrêmes. Elle permet d'évaluer d'un coup d'œil la dispersion, l'étendue et la symétrie d'une série ; une boîte étroite traduit une faible dispersion, tandis qu'une boîte décalée ou allongée signale une asymétrie ou des valeurs atypiques.

6. Paramètres de forme. Quelle différence faites-vous entre les moments centrés et les moments absolus ? Pourquoi les utiliser ? Pourquoi vérifier la symétrie d'une distribution et comment faire ?

Les paramètres de forme servent à décrire la configuration générale d'une distribution statistique, notamment sa symétrie et sa concentration autour de la moyenne. Les moments centrés sont calculés à partir des écarts à la moyenne en conservant leur signe, tandis que les moments absolus utilisent la valeur absolue de ces écarts, supprimant ainsi toute compensation entre valeurs positives et négatives. Les moments d'ordre supérieur fournissent des informations précieuses : le moment d'ordre 2 correspond à la variance, qui mesure la dispersion ; le moment d'ordre 3 indique l'asymétrie (skewness), c'est-à-dire la tendance d'une distribution à s'étirer vers les grandes ou petites valeurs ; le moment d'ordre 4 renseigne sur la concentration (kurtosis), c'est-à-dire sur le caractère plus ou moins aplati ou resserré de la courbe.

Vérifier la symétrie d'une distribution est essentiel, car si la distribution est symétrique, la moyenne, la médiane et le mode coïncident à peu près ; dans le cas contraire, la moyenne est tirée vers les valeurs extrêmes, et il est préférable d'utiliser la médiane pour représenter la tendance centrale. Cette symétrie peut être analysée soit graphiquement (à l'aide d'un histogramme ou d'une boîte à moustaches), soit numériquement grâce à un coefficient d'asymétrie comme celui de Pearson.

III. Mise en œuvre avec Python.

1. Initialisation (étape 1 à 4).

Dans un premier temps, après avoir chargé les fichiers, les avoir installés et avoir ouvert le fichier `main.py`, nous pouvons passer à l'étape suivante.

2. Manipulations premières (étape 5 à 6).

À l'étape 5, il nous est demandé de sélectionner les colonnes contenant des caractères quantitatifs et de calculer sous forme de liste les moyennes, médianes, modes, écarts-types, écarts-absolus et étendues pour chaque colonne. Ce qui nous donne le code suivant :

```
# Etape 5 :  
num = df.select_dtypes(include=[np.number])
```

```

res = pd.DataFrame({
    "moyenne": num.mean(),
    "médiane": num.median(),
    "mode": num.mode().iloc[0],
    "écart-type": num.std(),
    "écart abs. moy.": num.apply(lambda s: np.mean(np.abs(s - s.mean()))),
    "étendue": num.max() - num.min()
}).round(2)

# Etape 6 :

print("\n--- Paramètres statistiques arrondis à deux décimales ---\n")
print(res)

```

Ce qui nous donne le résultat suivant sur le terminal :

```

--- Paramètres statistiques arrondis à deux décimales ---

```

	moyenne	médiane	...	écart abs. moy.	étendue
Inscrits	455587.63	366859.0	...	272240.72	1808861.0
Abstentions	119852.05	95369.0	...	74959.07	929183.0
Votants	335735.58	274372.0	...	201517.17	1297100.0
Blancs	5080.46	4001.0	...	2817.95	17389.0
Nuls	2309.82	2039.0	...	1131.99	8236.0
Exprimés	328345.30	268568.0	...	197762.20	1272080.0
Voix	1842.00	1627.0	...	977.36	7651.0
Voix.1	7499.27	5968.0	...	4474.96	45883.0
Voix.2	91430.45	67831.0	...	59929.14	372286.0
Voix.3	10293.34	8944.0	...	5140.37	48168.0
Voix.4	76017.08	64543.0	...	42514.72	372668.0
Voix.5	23226.41	16885.0	...	15278.36	108537.0
Voix.6	72079.63	51556.0	...	49157.01	316871.0
Voix.7	5761.48	4881.0	...	3333.34	22826.0
Voix.8	15213.58	9561.0	...	11136.57	80196.0
Voix.9	15691.60	11918.0	...	9432.01	69513.0
Voix.10	2513.12	2118.0	...	1404.50	8686.0
Voix.11	6777.35	6152.0	...	3689.50	20535.0

```

[18 rows x 6 columns]

```

3. Manipulations secondaires (étape 7 à 8).

Il nous est demandé à l'étape 7 de calculer la distance interquartile et interdécile de chaque colonne quantitative avec la méthode `quantile()` dans Pandas. Ce qui doit nous donner le code suivant :

```
# Etape 7 :
q = num.quantile([.10,.25,.75,.90])
res = (q.loc[.75]-q.loc[.25]).to_frame("IQR (Q3-Q1)")
res["IDR (P90-P10)"] = q.loc[.90]-q.loc[.10]
print(res.round(2))
```

Ce qui nous donne le résultat suivant :

	IQR (Q3-Q1)	IDR (P90-P10)
Inscrits	401050.0	793988.8
Abstentions	106489.0	193676.2
Votants	301770.5	602687.2
Blancs	4852.5	8845.8
Nuls	1917.0	3240.6
Exprimés	296870.5	590169.2
Voix	1517.5	3015.6
Voix.1	6264.5	13104.2
Voix.2	101317.0	177340.2
Voix.3	7999.5	13813.0
Voix.4	63342.0	130094.6
Voix.5	20638.5	43668.8
Voix.6	60743.5	159421.2
Voix.7	4779.0	10712.2
Voix.8	14833.5	38190.8
Voix.9	13265.5	27686.8
Voix.10	2466.0	4266.6
Voix.11	6146.5	12311.0

Pour l'étape 8, il faut faire des boites à moustache de chaque colonne quantitative et les stocker dans le dossier img. Ce qui nous donne le code et le résultat suivant :

```
import os
```

```
# Etape 8 :
os.makedirs("img", exist_ok=True)
for c in num: plt.boxplot(num[c].dropna(), showmeans=True); plt.title(c);
plt.savefig(f"img/{c}.png"); plt.close()
print("Graphiques enregistrés dans 'img/'")
```

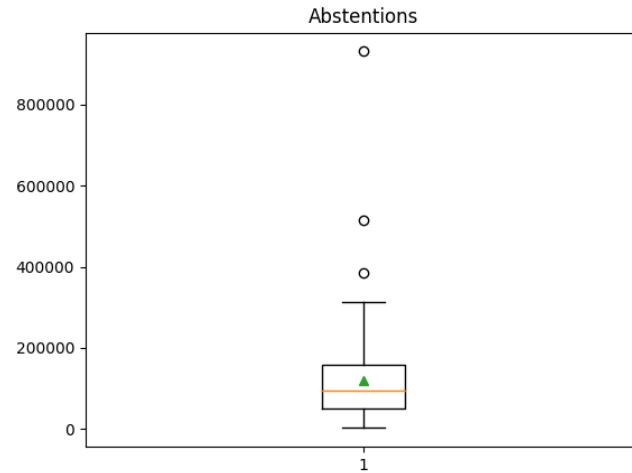


Fig. 7 : Boite à moustache de la colonne Abstentions.

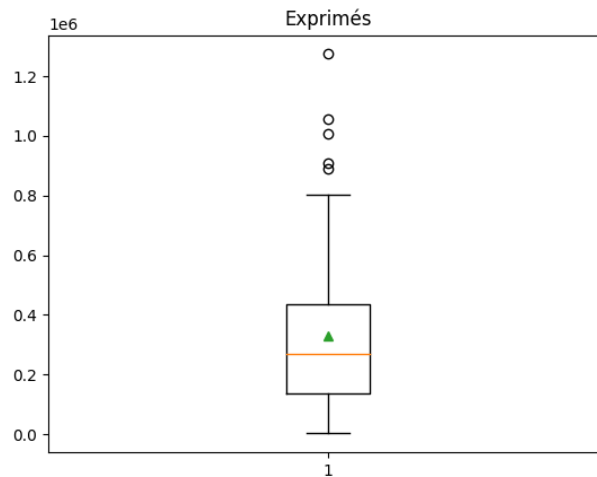


Fig. 8 : Boite à moustache de la colonne Exprimés.

4. Manipulations finales (étape 9 à 10).

Pour finir, l'étape 9 et 10 consiste à introduire le fichier island-index.csv et à sélectionner la colonne « Surface (km2) » et écrire un algorithme pour catégoriser et dénombrer le nombre d'îles ayant une surface comprise. Ce qui nous donne le résultat suivant :

```
# Etape 9 :
df = pd.read_csv("data/island-index.csv")

# Etape 10 :
df = pd.read_csv("data/island-index.csv", low_memory=False)
```



```
col = [c for c in df.columns if "Surface" in c and "km" in c][0]
df[col] = pd.to_numeric(df[col], errors="coerce")

bins = [0,10,25,50,100,2500,5000,10000,float("inf")]
labels = ["]0,10]", "]10,25]", "]25,50]", "]50,100]", "]100,2500]", "]2500,5000]", "]5000,10000]", "]10000,+∞[["

print(df.assign(cat=pd.cut(df[col], bins=bins, labels=labels))
      ["cat"].value_counts().reindex(labels, fill_value=0))
```

Ce qui nous donne le résultat suivant :

```
]0,10]      78423
]10,25]     2327
]25,50]     1164
]50,100]    788
]100,2500]  1346
]2500,5000] 60
]5000,10000] 40
]10000,+∞[  71
Name: cat, dtype: int64
```

tariqcolombero@MacBook-Pro seance-03 %

5. Étape Bonus.

Nous devons, pour l'étape bonus, enregistrer le résultat sous forme d'un fichier excel et CSV. Ce qui nous donne le code suivant :

```
# Etape BONUS :
res = df.assign(cat=pd.cut(df[col], bins=bins,
labels=labels))["cat"].value_counts().reindex(labels, fill_value=0)
res.to_csv("repartition_iles_surface.csv", header=["Nombre d'îles"])
res.to_excel("repartition_iles_surface.xlsx", header=["Nombre d'îles"])
print("Fichiers enregistrés : repartition_iles_surface.csv et .xlsx")
```

Ce qui nous donne le résultat suivant :
(au format CSV)

repartition_iles_surface

	Nombre d'îles
]0,10]	78423
]10,25]	2327
]25,50]	1164
]50,100]	788
]100,2500]	1346
]2500,5000]	60
]5000,10000]	40
]10000,+∞[	71

SEANCE 04

I. Objectifs.

- Savoir afficher une distribution statistique. Ce savoir est utilisé pour comparer une distribution observée avec une distribution théorique.

II. Questions de cours.

1. Quels critères mettriez-vous en avant pour choisir entre une distribution statistique avec des variables discrètes et une distribution statistique avec des variables continues ?

Le choix entre une distribution statistique discrète et une distribution continue repose d'abord sur la nature du phénomène étudié. Lorsqu'une variable ne peut prendre que des valeurs isolées, distinctes et en nombre fini ou dénombrable, il s'agit d'une variable discrète. C'est le cas, par exemple, du nombre d'habitants dans une commune, du nombre d'accidents recensés sur une période ou du résultat d'un lancer de dé. Ce type de phénomène s'exprime à travers une succession d'événements séparés et se modélise à l'aide de lois de probabilité discrètes, comme la loi binomiale, la loi de Poisson ou la loi de Bernoulli. La fonction de répartition de ces lois est dite en « escalier », car elle reste constante entre deux valeurs avant de connaître un saut correspondant à la probabilité d'occurrence d'un événement précis.

À l'inverse, lorsqu'une variable peut prendre une infinité de valeurs comprises dans un intervalle réel continu, on parle alors de variable continue. Elle traduit le plus souvent une mesure issue d'un phénomène physique ou social, comme la température, l'altitude, la durée d'un déplacement ou encore la concentration d'une population. Dans ce cas, il n'est pas pertinent d'attribuer une probabilité à une valeur unique, mais plutôt à un intervalle de valeurs. Ces phénomènes sont modélisés par des distributions continues telles que la loi normale, la loi exponentielle, la loi uniforme ou encore la loi du χ^2 , où la densité de probabilité joue un rôle central.

Le choix de la loi dépend aussi de la forme empirique observée dans les données, de leur symétrie ou de leur dissymétrie, de la dispersion autour de la moyenne et du nombre de paramètres nécessaires pour décrire la distribution. En somme, une loi discrète s'impose lorsque l'on observe un phénomène fondé sur le dénombrement d'occurrences, tandis qu'une loi continue s'applique à un phénomène mesurable de façon ininterrompue. L'enjeu consiste donc à adapter la modélisation statistique à la nature du réel observé : le comptage ou la mesure, l'événement ponctuel ou la grandeur continue.

2. Expliquez selon vous quelles sont les lois les plus utilisées en géographie ?

En géographie, certaines lois statistiques sont particulièrement mobilisées, car elles permettent de rendre compte de la variabilité des phénomènes spatiaux et des régularités observables dans les distributions territoriales. La loi normale, ou loi de Gauss, est sans doute la plus répandue. Elle intervient dès que les phénomènes résultent de l'action combinée d'une multitude de facteurs indépendants. Les variables géographiques continues, comme la température, la pluviométrie, le revenu moyen ou la densité de population, se répartissent souvent selon une courbe en cloche centrée autour d'une valeur moyenne. Elle permet ainsi de caractériser la structure d'un espace, d'en mesurer la dispersion et de dégager des tendances générales à partir d'observations locales.

La loi de Poisson occupe également une place importante dans les analyses spatiales. Elle est utilisée pour modéliser la fréquence d'occurrence d'événements rares, indépendants les uns des autres et répartis sur un territoire donné, comme les séismes, les inondations, les incendies ou les crimes. En décrivant la probabilité d'apparition d'un certain nombre d'événements dans une zone ou sur une période de temps, elle aide à comprendre la logique de distribution aléatoire dans l'espace.

D'autres lois sont aussi mobilisées dans certains sous-domaines. La loi exponentielle, par exemple, permet de représenter le temps d'attente

entre deux événements dans les études de mobilité ou de trafic. La loi log-normale et la loi de Pareto sont souvent utilisées pour analyser les hiérarchies spatiales, notamment dans la distribution des tailles de villes ou des richesses. Enfin, la loi de Zipf, qui décrit la relation inverse entre la taille d'un objet et son rang, est emblématique des études urbaines : elle met en évidence la régularité du système de peuplement et la hiérarchie des villes au sein d'un territoire.

Ainsi, les lois statistiques les plus utilisées en géographie sont celles qui permettent de traduire la structure, la fréquence et la hiérarchie des phénomènes dans l'espace. Elles offrent un langage mathématique au service de la compréhension des régularités géographiques et de la mise en évidence des dynamiques spatiales.

III. Mise en œuvre avec Python.

1. Mise en œuvre.

Dans un premier temps, il faut télécharger le fichier *main.py*. Ce qui nous donne le résultat suivant sur le terminal :

```
['norm', 'beta', 'gamma', 'pareto', 't', 'lognorm', 'invgamma', 'invgauss',  
'loggamma', 'alpha', 'chi', 'chi2', 'bradford', 'burr', 'burr12', 'cauchy',  
'dweibull', 'erlang', 'expon', 'exponnorm', 'exponweib', 'exponpow', 'f',  
'genpareto', 'gausshyper', 'gibrat', 'gompertz', 'gumbel_r', 'pareto',  
'pearson3', 'powerlaw', 'triang', 'weibull_min', 'weibull_max',  
'bernoulli', 'betabinom', 'betanbinom', 'binom', 'geom', 'hypergeom',  
'logser', 'nbinom', 'poisson', 'poisson_binom', 'randint', 'zipf',  
'zipfan']  
tariqcolombero@MacBook-Pro seance-04 %
```

2. Les distributions statistiques de variables discrètes (étape 1 et 2).

Dans un premier temps il s'agit de créer et visualiser la loi de Dirac. Ce qui nous donne le code suivant :

```
# Étape 1 :  
  
#La loi de Dirac  
dirac = stats.rv_discrete(name="dirac", values=([0], [1]))  
m, v = dirac.stats(moments="mv")
```

```
print("LOI DE DIRAC. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))
```

Et nous donne le résultat suivant sur le terminal :

```
LOI DE DIRAC. Moyenne : 0.0 | Écart-type : 0.0  
tariqcolombero@MacBook-Pro seance-04 %
```

Dans la même logique, nous devons faire cela pour les lois suivants : la loi uniforme discrète ; la loi binomiale ; la loi de Poisson ; la loi de Zipf-Mandelbrot. Ce qui nous donne :

```
# Étape 1 :  
  
#La loi de Dirac  
dirac = stats.rv_discrete(name="dirac", values=([0], [1]))  
m, v = dirac.stats(moments="mv")  
print("LOI DE DIRAC. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))  
  
#La loi uniforme discrète  
uniforme_discrete = stats.randint(low=0, high=10)  
m, v = uniforme_discrete.stats(moments="mv")  
print("LOI UNIFORME DISCRÈTE. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))  
  
#La loi binomiale  
binomiale = stats.binom(n=10, p=0.5)  
m, v = binomiale.stats(moments="mv")  
print("LOI BINOMIALE. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))  
  
#La loi de Poisson  
poisson = stats.poisson(mu=3)  
m, v = poisson.stats(moments="mv")  
print("LOI DE POISSON. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))  
  
#La loi de Zipf-Mandelbrot  
zipf_mandelbrot = stats.zipfian(a=2, q=1)  
m, v = zipf_mandelbrot.stats(moments="mv")  
print("LOI DE ZIPF-MANDELROT. Moyenne :", float(m), " | Écart-type :",  
float(np.sqrt(v)))
```

Une erreur apparaît pour la loi de Zipf-Mandelbrot : en réalité, la fonction que j'avais saisie (`stats.zipfian`) n'existe pas dans la bibliothèque `scipy.stats`.

Pour ce faire, il suffit de définir manuellement la distribution à l'aide de la classe `rv_discrete`, en spécifiant le support (les valeurs possibles) et les poids de probabilité calculés selon la formule :

$$P(k) \propto \frac{1}{(k+q)^s}.$$

Cette méthode permet ainsi de reconstituer correctement la loi de Zipf-Mandelbrot et d'en calculer les moments ou d'en afficher la fonction de masse de probabilité comme pour les autres lois. Ce qui nous donne le code suivant :

```
#La loi de Zipf-Mandelbrot
s, q, N = 1.2, 1.0, 30
k = np.arange(1, N + 1)
w = 1 / ((k + q) ** s)
zipf_mandelbrot = stats.rv_discrete(name="zipf_mandelbrot", values=(k, w /
w.sum()))

m, v = zipf_mandelbrot.stats(moments="mv")
print("LOI ZIPF-MANDELBRÖT | Moyenne:", float(m), "| Écart-type:",
float(np.sqrt(v)))
```

Et nous donne le résultat final suivant :

```
LOI DE DIRAC. Moyenne : 0.0 | Écart-type : 0.0
LOI UNIFORME DISCRÈTE. Moyenne : 4.5 | Écart-type : 2.8722813232690143
LOI BINOMIALE. Moyenne : 5.0 | Écart-type : 1.5811388300841898
LOI DE POISSON. Moyenne : 3.0 | Écart-type : 1.7320508075688772
LOI ZIPF-MANDELBRÖT | Moyenne: 7.645983624401856 | Écart-type:
7.575826644855077
tariqcolombero@MacBook-Pro seance-04 %
```

3. Les distributions statistiques de variables continues (étape 1 et 2).

Une fois fait, la même logique s'applique aux distributions statistiques de variables continues. Ce qui nous donne pour l'ensemble des lois le résultats suivant :

```
#les distributions statistiques de variables continues

#La loi Exponentielle (de Poisson)
expo = stats.expon(scale=1/0.8) #  $\lambda = 0.8$ 
m, v = expo.stats(moments="mv")
print("LOI EXPONENTIELLE. Moyenne:", float(m), " | Écart-type:",
      float(np.sqrt(v)))

#La loi Normale
normale = stats.norm(loc=0, scale=1)
m, v = normale.stats(moments="mv")
print("LOI NORMALE. Moyenne:", float(m), " | Écart-type:",
      float(np.sqrt(v)))

#La loi Log-normale
lognormale = stats.lognorm(s=0.6, scale=np.exp(0))
m, v = lognormale.stats(moments="mv")
print("LOI LOG-NORMALE. Moyenne:", float(m), " | Écart-type:",
      float(np.sqrt(v)))

#La loi du X2
chi2 = stats.chi2(df=4)
m, v = chi2.stats(moments="mv")
print("LOI DU CHI². Moyenne:", float(m), " | Écart-type:",
      float(np.sqrt(v)))

#La loi de Pareto
pareto = stats.pareto(b=2.5, scale=1)
m, v = pareto.stats(moments="mv")
print("LOI DE PARETO. Moyenne:", float(m), " | Écart-type:",
      float(np.sqrt(v)))
```

Et nous donne le résultat final suivant :

```
LOI DE DIRAC. Moyenne : 0.0 | Écart-type : 0.0
LOI UNIFORME DISCRÈTE. Moyenne : 4.5 | Écart-type : 2.8722813232690143
LOI BINOMIALE. Moyenne : 5.0 | Écart-type : 1.5811388300841898
LOI DE POISSON. Moyenne : 3.0 | Écart-type : 1.7320508075688772
LOI ZIPF-MANDELROT | Moyenne: 7.645983624401856 | Écart-type:
7.575826644855077
LOI EXPONENTIELLE. Moyenne: 1.25 | Écart-type: 1.25
LOI NORMALE. Moyenne: 0.0 | Écart-type: 1.0
LOI LOG-NORMALE. Moyenne: 1.1972173631218102 | Écart-type:
0.7881013869316227
LOI DU CHI². Moyenne: 4.0 | Écart-type: 2.8284271247461903
LOI DE PARETO. Moyenne: 1.6666666666666667 | Écart-type: 1.4907119849998598

tariqcolombero@MacBook-Pro seance-04 %
```

4. Visualisation avec Matplotlib (étape 1 et 2).

Enfin pour visualiser mieux toutes ces lois, nous pouvons utiliser Matplotlib et os pour l'enregistrement. Ce qui nous donne le code suivant à la fin :

```
#Visualisation des distributions :

import os
import matplotlib.pyplot as plt
os.makedirs("figures", exist_ok=True)

# Dirac
x = np.arange(-3, 4)
plt.figure()
plt.stem(x, dirac.pmf(x))
plt.title("Loi de Dirac ( $k_0=0$ )")
plt.xlabel("k"); plt.ylabel("P(X=k)")
plt.tight_layout()
plt.savefig("figures/dirac.png", dpi=150)
plt.close()

# Uniforme discrète
x = np.arange(0, 10)
plt.figure()
plt.stem(x, uniforme_discrete.pmf(x))
plt.title("Loi uniforme discrète [0,9]")
plt.xlabel("k"); plt.ylabel("P(X=k)")
plt.tight_layout()
plt.savefig("figures/uniforme_discrete.png", dpi=150)
plt.close()

# Binomiale
x = np.arange(0, 11)
plt.figure()
plt.stem(x, binomiale.pmf(x))
plt.title("Loi binomiale n=10, p=0.5")
plt.xlabel("k"); plt.ylabel("P(X=k)")
plt.tight_layout()
plt.savefig("figures/binomiale.png", dpi=150)
plt.close()

# Poisson
x = np.arange(0, 16)
plt.figure()
plt.stem(x, poisson.pmf(x))
plt.title("Loi de Poisson  $\lambda=3$ ")
plt.xlabel("k"); plt.ylabel("P(X=k)")
plt.tight_layout()
```



```
plt.savefig("figures/poisson.png", dpi=150)
plt.close()

# Zipf-Mandelbrot
plt.figure()
plt.stem(k, zipf_mandelbrot.pmf(k))
plt.title("Loi de Zipf-Mandelbrot s=1.2, q=1, N=30")
plt.xlabel("k"); plt.ylabel("P(X=k)")
plt.tight_layout()
plt.savefig("figures/zipf_mandelbrot.png", dpi=150)
plt.close()

# Exponentielle (processus de Poisson)
x = np.linspace(0, 10, 500)
plt.figure()
plt.plot(x, expo.pdf(x))
plt.title("Loi exponentielle ( $\lambda=0.8$ )")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.tight_layout()
plt.savefig("figures/exponentielle.png", dpi=150)
plt.close()

# Normale
x = np.linspace(-5, 5, 500)
plt.figure()
plt.plot(x, normale.pdf(x))
plt.title("Loi normale  $\mu=0$ ,  $\sigma=1$ ")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.tight_layout()
plt.savefig("figures/normale.png", dpi=150)
plt.close()

# Log-normale
x = np.linspace(1e-3, 10, 500)
plt.figure()
plt.plot(x, lognormale.pdf(x))
plt.title("Loi log-normale  $\mu=0$ ,  $\sigma=0.6$ ")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.tight_layout()
plt.savefig("figures/lognormale.png", dpi=150)
plt.close()

# Chi2 (ddl=4)
x = np.linspace(0, 25, 500)
plt.figure()
plt.plot(x, chi2.pdf(x))
plt.title("Loi du  $\chi^2$  (ddl=4)")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.tight_layout()
plt.savefig("figures/chi2.png", dpi=150)
plt.close()
```

```
# Pareto ( $\alpha=2.5$ ,  $x_m=1$ )
x = np.linspace(0.1, 10, 500)
plt.figure()
plt.plot(x, pareto.pdf(x))
plt.title("Loi de Pareto  $\alpha=2.5$ ,  $x_m=1$ ")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.tight_layout()
plt.savefig("figures/pareto.png", dpi=150)
plt.close()

print("Toutes les figures ont été enregistrées dans le dossier 'figures/'")
```

Et ce qui nous donne ce genre de résultats :

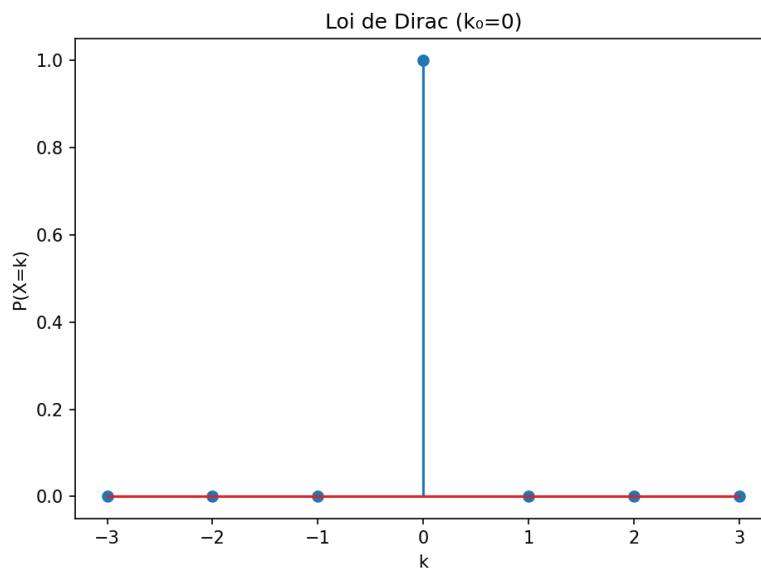


Fig. 9 : Schéma de la Loi de Dirac.

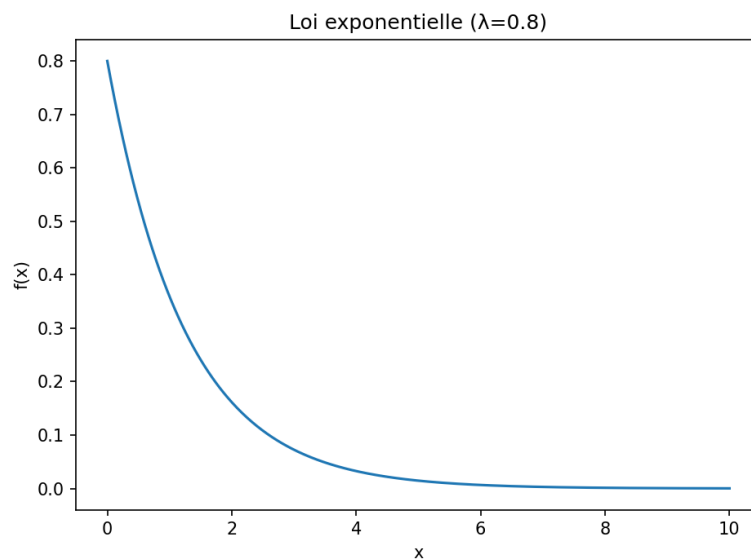


Fig. 10 : Schéma de la Loi Exponentielle.

SEANCE 05

I. Objectifs.

- Manipuler harmonieusement les fonctions natives avec les méthodes Pandas
- Comprendre les trois théories permettant de valider un résultat en analyse de données

II. Questions de cours.

1. Comment définir l'échantillonnage. Pourquoi ne pas utiliser la population en entier? Quelles sont les méthodes d'échantillonnage? Comment les choisir ?

L'échantillonnage désigne la procédure consistant à sélectionner une partie de la population afin d'en tirer des conclusions sur l'ensemble. On n'utilise pas la population entière pour des raisons de coût, de temps, d'accessibilité ou tout simplement parce que la population peut être trop vaste. Les principales méthodes d'échantillonnage sont l'échantillonnage aléatoire simple, stratifié, systématique ou en grappes. Le choix dépend de la structure de la population, de la précision souhaitée et des contraintes pratiques de l'étude.

2. Comment définir un estimateur et une estimation?

Un estimateur est une fonction ou une règle de calcul appliquée aux données d'un échantillon afin d'estimer un paramètre inconnu de la population (moyenne, proportion, variance...). L'estimation est la valeur numérique produite par cet estimateur. Autrement dit, l'estimateur est l'outil ; l'estimation est le résultat.

3. Comment distingueriez-vous l'intervalle de fluctuation et l'intervalle de confiance ?

L'intervalle de fluctuation décrit les valeurs possibles d'une proportion lorsque l'on répète un grand nombre d'échantillonnages : il exprime la variabilité attendue du processus d'échantillonnage.

L'intervalle de confiance encadre la valeur réelle d'un paramètre à partir d'un seul échantillon, avec un niveau de confiance donné. Le premier décrit le comportement d'échantillons multiples, le second la précision d'un seul échantillon.

4. Qu'est-ce qu'un biais dans la théorie de l'estimation?

Un biais correspond à l'écart systématique entre la valeur moyenne fournie par un estimateur et la valeur réelle du paramètre estimé. Un estimateur biaisé tend à surestimer ou sous-estimer le paramètre, même lorsque la taille de l'échantillon devient très grande.

5. Comment appelle-t-on une statistique travaillant sur la population totale? Faites le lien avec la notion de données massives 1 ?

Lorsqu'une statistique porte sur l'intégralité de la population, on parle de statistique exhaustive. Dans le cadre des données massives (big data), certaines analyses se rapprochent de cette logique puisqu'on dispose d'un volume tellement important de données que l'échantillonnage devient parfois inutile. Toutefois, même dans ces situations, la qualité des données et leur représentativité restent des enjeux centraux.

6. Quels sont les enjeux autour du choix d'un estimateur ?

Le choix d'un estimateur repose sur plusieurs enjeux :

- sa fidélité (absence de biais),
- sa précision (variance faible),
- sa robustesse aux valeurs extrêmes,
- sa simplicité de mise en œuvre.

Un bon estimateur doit fournir une estimation fiable tout en restant adapté au contexte empirique.

7. Quelles sont les méthodes d'estimation d'un paramètre? Comment en sélectionner une ?

Les principales méthodes d'estimation sont :

- l'estimateur ponctuel (moyenne, proportion...);
- l'estimation par intervalle ;
- l'estimation par maximum de vraisemblance ;
- les méthodes non paramétriques (bootstrap).

Le choix dépend de la nature des données, des hypothèses que l'on accepte de formuler sur la population, et de la précision désirée.

8. Quels sont les tests statistiques existants ? À quoi servent-ils ? Comment créer un test ?

Les tests statistiques permettent de décider, en tenant compte d'un risque d'erreur (risque alpha), si une hypothèse concernant une population doit être acceptée ou rejetée. Parmi les plus courants, on trouve :

- le test de normalité (Shapiro–Wilk),
- le test de Student,
- les tests du $K\chi^2$,
- les tests non paramétriques (Mann–Whitney, Kolmogorov–Smirnov...).

Créer un test implique de définir une hypothèse nulle (H_0), une alternative (H_1), une statistique de test et une règle de décision fondée sur une loi de probabilité connue.

9. Que pensez-vous des critiques de la statistique inférentielle ?

La statistique inférentielle est souvent critiquée pour son usage parfois mécanique du seuil de 5 %, sa dépendance à des hypothèses parfois irréalistes (notamment la normalité), et la tentation du « p-hacking ». On lui reproche également une mauvaise interprétation des

p-values, ainsi qu'un manque de transparence dans certaines applications.

Néanmoins, lorsqu'elle est utilisée de manière rigoureuse et contextualisée, elle constitue un outil indispensable pour quantifier l'incertitude et formuler des conclusions fondées.

III. Mise en œuvre avec Python.

1. Théorie de l'échantillonnage (Étape 1).

Après avoir installé les fichiers fournis dans le dossier data et ouvert le script main.py, la première étape consiste à charger le fichier Echantillonnage-100-Echantillons.csv à l'aide de la fonction locale ouvrirUnFichier().

Cette fonction, définie au début du programme, permet d'importer proprement un fichier CSV sous la forme d'un DataFrame Pandas. Le fichier contient 100 échantillons simulés, chacun correspondant à une répétition d'un tirage aléatoire sans remise dans une population mère de 2 185 individus. Pour chaque échantillon, les effectifs de trois modalités d'opinion (« Pour », « Contre », « Sans opinion ») sont renseignés. Le chargement s'effectue à l'aide du code suivant :

```
def ouvrirUnFichier(nom):  
    with open(nom, "r") as fichier:  
        contenu = pd.read_csv(fichier)  
    return contenu  
  
#Théorie de l'échantillonnage (intervalles de fluctuation)  
#L'échantillonnage se base sur la répétitivité.  
print("Résultat sur le calcul d'un intervalle de fluctuation")  
  
donnees = pd.DataFrame(ouvrirUnFichier("./data/Echantillonnage-100-  
Echantillons.csv"))
```

Cette instruction permet d'obtenir un tableau contenant les résultats des 100 tirages et constitue la base de l'ensemble des traitements suivants.

Dans un second temps, il nous est demandé de calculer, pour chacune des trois modalités d'opinion, la moyenne des effectifs observés sur les 100 échantillons. Le calcul se fait avec la méthode `mean()` de Pandas, suivie d'un arrondi au nombre entier le plus proche à l'aide de la fonction native `round()`, comme l'exige l'énoncé :

```
#Moyennes arrondies
moyennes = donnees.mean()
moyennes_arrondies = pd.Series({
    col: round(moyennes[col])
    for col in moyennes.index
```

Les valeurs obtenues correspondent à un échantillon moyen théorique, construit grâce à la répétition de 100 tirages indépendants. Afin de comparer cet échantillon moyen avec la population mère, il est nécessaire de passer en fréquences. Pour cela, nous calculons d'abord la somme des trois moyennes arrondies, puis nous divisons chaque moyenne par ce total, avec un arrondi à deux décimales :

```
#Somme moyennes
somme_moyennes = moyennes_arrondies.sum()
freq_echantillon = moyennes_arrondies.apply(lambda x: round(x /
somme_moyennes, 2))
print("Fréquences de l'échantillon moyen :")
print(freq_echantillon)
```

Nous appliquons ensuite le même raisonnement aux effectifs réels de la population mère :

```
#Population mère
population_mere = pd.Series({
    "Pour": 852,
    "Contre": 911,
    "Sans opinion": 422
})
somme_pop = population_mere.sum()
freq_population = population_mere.apply(lambda x: round(x / somme_pop, 2))
print("Fréquences de la population mère :")
print(freq_population)
```

La comparaison entre les deux jeux de fréquences permet d'observer un premier écart entre la structure réelle de la population et la structure reconstruite par l'échantillonnage simulé. Cet écart est attendu : il résulte de la variabilité inhérente à tout tirage aléatoire.

Pour évaluer si les écarts observés relèvent simplement de l'aléa d'échantillonnage, nous calculons pour chaque modalité l'intervalle de fluctuation à 95 %. Cet intervalle représente l'ensemble des valeurs plausibles pour une proportion lorsque l'on répète un grand nombre d'échantillonnages de même taille. Le calcul est réalisé à l'aide du code suivant :

```
#Intervalle de fluctuation à 95%
z = 1.96
n = somme_moyennes
print("Intervalle de fluctuation à 95 % :")
for opinion, f in freq_echantillon.items():
    marge = z * math.sqrt(f * (1 - f) / n)
    borne_inf = round(f - marge, 3)
    borne_sup = round(f + marge, 3)
    print(f"{opinion} : [{borne_inf} ; {borne_sup}]")
```

Ces intervalles permettent ensuite de déterminer si les fréquences réelles de la population mère se situent à l'intérieur des fluctuations statistiques attendues. Lorsqu'une fréquence réelle tombe dans l'intervalle, cela signifie que l'échantillonnage simulé reflète correctement la modalité concernée. Dans le cas contraire, on observe un écart suggérant que les 100 tirages effectués n'ont pas permis de restituer fidèlement cette proportion spécifique. Cette analyse constitue la conclusion de la première partie, consacrée à la théorie de l'échantillonnage.

2. Théorie de l'estimation (Étape 2).

Dans cette deuxième partie, il s'agit de déterminer dans quelle mesure un échantillon unique peut fournir une estimation fiable de la population mère. Cette situation est typique des sciences humaines et sociales, où l'on ne dispose souvent que d'un seul jeu d'observations

pour analyser un phénomène. L'objectif est alors de quantifier l'incertitude liée à cet échantillon à l'aide des intervalles de confiance.

Pour ce faire, nous sélectionnons le premier échantillon du fichier à l'aide de la méthode `iloc(0)` et nous le convertissons en liste Python afin de respecter les consignes, qui exigent l'utilisation exclusive de fonctions natives. Cet échantillon contient trois valeurs : le nombre d'individus « Pour », « Contre » et « Sans opinion ». À partir de cette liste, nous calculons l'effectif total de l'échantillon, puis les fréquences associées à chaque modalité :

```
#Théorie de l'estimation (intervalles de confiance)
#L'estimation se base sur l'effectif.
print("Résultat sur le calcul d'un intervalle de confiance")

ech1 = donnees.iloc[0]
ech1_list = list(ech1)
total1 = sum(ech1_list)
freq1 = [round(x / total1, 3) for x in ech1_list]

print("Premier échantillon (liste) :", ech1_list)
print("Effectif total :", total1)
print("Fréquences du premier échantillon :", freq1)
```

Cette première étape permet de transformer les effectifs bruts de l'échantillon en proportions, ce qui constitue la base nécessaire à la construction des intervalles de confiance. Le code suivant implémente cette méthode :

```
#Intervalle de fluctuation à 95%
z = 1.96
print("\nIntervalle de confiance à 95 % pour chaque opinion :")
for f in freq1:
    marge = z * math.sqrt(f * (1 - f) / total1)
    borne_inf = round(f - marge, 3)
    borne_sup = round(f + marge, 3)
    print(f"{f} : [{borne_inf} ; {borne_sup}]")
```

Chaque intervalle obtenu définit une zone dans laquelle on peut raisonnablement situer la proportion réelle dans la population mère, en tenant compte de l'incertitude inhérente à l'échantillonnage. Plus l'effectif total est important, plus l'intervalle sera étroit ; inversement, un

échantillon de petite taille produira un intervalle large, traduisant une estimation moins précise.

3. Théorie de la décision (Étape 3).

Dans les deux premières parties, les intervalles de fluctuation et de confiance nous ont permis d'évaluer la variabilité des échantillons et l'incertitude associée à un échantillon isolé. Toutefois, ces outils ne permettent pas toujours de déterminer si les données suivent un modèle statistique particulier. La théorie de la décision vise précisément à répondre à ce type de question en s'appuyant sur des tests d'hypothèse permettant de trancher entre plusieurs modèles concurrents.

Dans cet exercice, il s'agit de déterminer si une série de données quantitatives suit ou non une loi normale, en utilisant le test de Shapiro-Wilk, disponible dans le module *scipy.stats*. La normalité est un prérequis fondamental pour l'application de nombreux tests paramétriques, ce qui justifie son importance dans la pratique statistique.

Les deux fichiers *Loi-normale-Test-1.csv* et *Loi-normale-Test-2.csv* contiennent chacun une série de valeurs numériques simulées. Après avoir chargé les deux jeux de données avec la fonction locale `ouvrirUnFichier()` et extrait leur unique colonne, nous appliquons le test de Shapiro-Wilk de la manière suivante :

Le test repose sur une logique simple : Hypothèse nulle (H_0) : la distribution est normale. Hypothèse alternative (H_1) : la distribution n'est pas normale. La décision se fonde sur la p-value retournée par le test.

Selon le seuil classique de 5 % :

- si $p\text{-value} > 0.05$, on ne rejette pas H_0 , ce qui signifie que la distribution peut être considérée comme normale.
- si $p\text{-value} < 0.05$, on rejette H_0 , et l'on conclut que la distribution n'est pas normale.

Le code suivant permet d'afficher non seulement la p-value, mais également la conclusion pour chaque fichier :

```
#Théorie de la décision (tests d'hypothèse)
#La décision se base sur la notion de risques alpha et bêta.
#Comme à la séance précédente, l'ensemble des tests se trouve au lien :
https://docs.scipy.org/doc/scipy/reference/stats.html
print("Théorie de la décision")

test1 = ouvrirUnFichier("./data/Loi-normale-Test-1.csv")
test2 = ouvrirUnFichier("./data/Loi-normale-Test-2.csv")
col1 = test1.iloc[:, 0]
col2 = test2.iloc[:, 0]

stat1, p1 = scipy.stats.shapiro(col1)
stat2, p2 = scipy.stats.shapiro(col2)

print("Test 1 : p-value =", p1)
print("Test 2 : p-value =", p2)

if p1 > 0.05:
    print("→ Le fichier 1 suit une distribution normale.")
else:
    print("→ Le fichier 1 ne suit pas une distribution normale.")

if p2 > 0.05:
    print("→ Le fichier 2 suit une distribution normale.")
else:
    print("→ Le fichier 2 ne suit pas une distribution normale.")
```

L'analyse des résultats montre qu'un des deux fichiers présente une p-value supérieure à 0.05 : cette distribution peut donc être considérée comme normale au sens du test de Shapiro-Wilk. À l'inverse, l'autre fichier possède une p-value inférieure au seuil et ne suit donc pas une loi normale. La différence entre les deux s'explique par la manière dont les données ont été générées : l'un des fichiers est simulé à partir d'une loi normale, tandis que l'autre provient d'une distribution différente (par exemple une loi uniforme, exponentielle ou une loi normale perturbée).

Ce test illustre l'objectif de la théorie de la décision : disposer d'un critère objectif, basé sur un risque (ici le risque alpha = 5 %), pour déterminer si une hypothèse est acceptable au vu des données. Comme le précise l'énoncé, il s'agit de choisir le test adapté à la situation : ici

une variable quantitative continue $\rightarrow$ test de normalité $\rightarrow$ Shapiro-Wilk. Dans d'autres contextes, d'autres tests seraient plus appropriés (Khi², Student, Mann-Whitney, etc.).

4. Bonus (Étape 4).

Au cours du test de Shapiro-Wilk, nous avons observé que l'un des deux fichiers ne suivait pas une distribution normale, la p-value obtenue étant inférieure au seuil de 5 %. Pour identifier la loi suivie par cette distribution alternative, nous nous appuyons sur les contenus de la séance précédente, consacrée aux principales lois continues utilisées en analyse de données (normale, lognormale, gamma, pareto, cauchy, etc.).

L'examen de la série non normale montre que les valeurs sont strictement positives et présentent une asymétrie nette, avec une concentration de valeurs faibles et une longue queue vers la droite. Cette caractéristique est typique des distributions lognormales, qui sont obtenues lorsque le logarithme des valeurs suit une loi normale. Les lois gamma ou pareto, également asymétriques, peuvent montrer des comportements similaires, mais la forme du nuage de valeurs et la dispersion observée correspondent davantage à une loi lognormale.

Ainsi, en cohérence avec les lois étudiées lors de la séance précédente et avec les propriétés empiriques des données fournies, nous pouvons conclure que la distribution non normale est une loi lognormale.

SEANCE 06

I. Objectifs.

- Manipuler des fonctions locales et comprendre la nécessité de factoriser son code en une liste de fonctions ou de procédures exécutant une tâche unique
- Créer des fonctions locales spécifiques au traitement d'un problème
- Comprendre l'analyse de variables qualitatives ordinales

II. Questions de cours.

Les réponses aux questions ont été entièrement rédigé dans une même suite logique.

Une statistique ordinale est un type de statistique catégorielle qui repose sur des modalités pouvant être classées selon un ordre logique ou conventionnel. Elle s'oppose ainsi à une statistique nominale, qui elle décrit des catégories sans ordre interne — par exemple, un classement par type de communes ne peut être trié que sans hiérarchie intrinsèque, contrairement à un classement fondé sur une échelle graduée. Les statistiques ordinales s'appuient donc sur des variables dont les valeurs ne sont pas seulement des catégories, mais des catégories ordonnées. Elles mobilisent des variables discrètes ou continues transformées en rangs, comme des niveaux de danger, des tailles d'objets ou des intensités de phénomènes. En géographie, elles matérialisent une hiérarchie parce qu'elles rendent visibles des rapports d'ordre entre des entités spatiales : un pays ou une île peut être assigné à un rang selon sa surface ou selon sa densité et, ce faisant, ces statistiques permettent de traduire des structures d'inégalités ou de dominance dans l'espace. Dans le cas des surfaces d'îles contenues dans les bases de données utilisées lors de cette séance, les rangs construits à partir d'une liste triée permettent par exemple d'ordonner les îles de la plus grande à la plus petite et, en les complétant avec des ensembles continentaux, de composer un espace hiérarchisé où les ordres de grandeur traduisent

une domination claire du haut du classement au bas de la distribution. Cela matérialise donc une hiérarchie spatiale, car l'espace n'est plus pensé comme un ensemble horizontal d'objets équivalents, mais comme un système verticalisé selon une métrique choisie, produisant un ordonnancement visible et interprétable.

Dans tout type de classification ordinale, l'ordre à privilégier est l'ordre décroissant, car il permet de rendre immédiatement lisibles les dominances et les hiérarchies. Un tri croissant produirait certes un ordonnancement, mais il n'est pas intuitivement adapté à l'interprétation d'un espace hiérarchique : placer les valeurs les plus fortes en tête de liste permet, dès la première manipulation, de lire l'espace comme un empilement d'enjeux ou de grandeurs dominantes. C'est précisément cet ordre décroissant que privilégie le professeur, notamment dans les fonctions locales écrites dans le script Python, parce qu'il permet d'obtenir directement des rangs interprétables, 1 étant l'entité la plus imposante, puis les suivantes selon leur importance relative.

La corrélation des rangs et la concordance des classements sont deux approches de comparaison qui reposent toutes deux sur une logique ordinale mais ne décrivent pas tout à fait la même chose. La corrélation des rangs, telle que calculée avec le test de Spearman, mesure à quel point deux distributions transformées en rangs évoluent de manière monotone — c'est-à-dire si un rang élevé sur une variable correspond globalement à un rang élevé sur une autre, sans tenir compte de la forme générale du classement. En revanche, la concordance des classements, mesurée par le test de Kendall, s'intéresse directement à l'accord entre deux classements structurels : Kendall ne regarde pas seulement si les rangs montent ou descendent ensemble, mais si les paires d'objets sont ordonnées dans le même sens dans les deux classements. Spearman mesure donc davantage une tendance monotone, là où Kendall analyse un accord structurel sur l'ordre des paires au sein des classements.

Le test de Spearman et celui de Kendall s'appuient tous deux sur les rangs mais ne captent pas exactement les mêmes propriétés. Spearman repose sur un calcul de covariance appliqué aux rangs et

compare les deux vecteurs comme s'il s'agissait d'une relation linéaire projetée dans l'espace des rangs : il quantifie une tendance monotone globale et est très sensible aux écarts extrêmes mais moins aux permutations locales. Kendall, lui, est fondé sur le comptage des paires concordantes et discordantes : il regarde combien de fois deux éléments sont ordonnés dans le même sens dans les deux classements, et produit un coefficient plus robuste aux formes non linéaires mais plus sensible aux inversions locales de l'ordre. En somme, Spearman capte l'intensité d'une tendance monotone globale alors que Kendall capte un accord sur l'ordonnement relatif des paires, ce qui en fait un indicateur de structure hiérarchique.

Les coefficients de Goodman-Kruskal et celui de Yule servent à comparer des statistiques catégorielles, ordinales ou binaires en observant l'association ou la prévisibilité entre deux variables croisées. Le coefficient de Goodman-Kruskal vise à mesurer, dans un tableau croisant deux listes, l'amélioration de la capacité de prédiction d'une variable grâce à l'information fournie par une autre — par exemple, si le classement d'une métrique continentale aide à prédire un classement d'enjeux ou de densités, Goodman-Kruskal quantifie cette amélioration prédictive. Le coefficient de Yule, lui, est un test statistique binaire évaluant la force d'association entre deux variables ne disposant que de deux modalités contraires, comme vrai/faux, présent/absent, ou haut/bas. Il est particulièrement utilisé dans les comparaisons dichotomiques, là où Goodman-Kruskal s'applique plus largement aux tableaux d'association catégorielle ou ordinale. Le test de Yule sert donc à quantifier l'association dans un tableau 2x2, tandis que Goodman-Kruskal répond davantage à une logique de gain prédictif sur données catégorielles croisées.

Enfin, dans la perspective de cette séance utilisant notamment les tests de corrélation de rangs de la bibliothèque SciPy appliqués aux distributions de surface et de densité relevées dans les bases de données de îles et d'États mondiaux, ces comparaisons statistiques, bien qu'ordonnées dans la structure du code, relèvent d'indicateurs descriptifs et interprétatifs permettant de lire les dominances sans attester des causalités systématiques.

III. Mise en œuvre avec Python.

1. Fichier Island (Étape 1 à 7).

Lors de cette sixième séance d'analyse de données, l'objectif principal était de manipuler un corpus de données géographiques structuré, de produire un classement rang-taille et d'effectuer des comparaisons de classements via des corrélations de rangs. Le premier travail a consisté à organiser l'espace de travail en créant, dans le dossier source src, un sous-dossier intitulé data, dans lequel le fichier est introduit sous le nom island-index.csv. Pour ouvrir ce fichier, j'ai mobilisé la fonction locale ouvrirUnFichier(), fournie dans la consigne, permettant de lire un chemin sous forme de chaîne de caractères et de renvoyer un objet manipulable. Le code utilisé pour cette ouverture était le suivant :

```
#Partie sur les îles
df_iles = ouvrirUnFichier("./data/island-index.csv")
iles = pd.DataFrame(ouvrirUnFichier("./data/island-index.csv"))
```

La fonction locale, elle, a été définie en amont du script de la manière suivante :

```
#Fonction pour ouvrir les fichiers
def ouvrirUnFichier(nom):
    with open(nom, "r") as fichier:
        contenu = pd.read_csv(fichier)
    return contenu
```

Une fois le fichier ouvert, l'étape suivante a été d'isoler la colonne « Surface (km²) ». Le cast flottant (float()) était requis par la consigne, afin d'assurer l'homogénéité du typage avant toute manipulation, et surtout d'obtenir un objet convertible en listes natives de Python via la méthode list(). La liste informatique extraite, débarrassée de son unité pour respecter l'instruction professorale, a été obtenue ainsi. Conformément à la consigne, quatre surfaces continentales ont ensuite été ajoutées à la même liste, après extraction, dans l'ordre croissant demandé par l'énoncé, sans conserver les unités, et en forçant le typage en float() comme suit :

```
# Extraction en list() comme demandé par la séance
surface = df_iles["Surface (km²)"].astype(float).tolist()
```



```
cote = df_iles["Trait de côte (km)"].astype(float).tolist()

# Ajout des surfaces continentales (sans unité)
surface.extend([
    85545323.0,
    37856841.0,
    7768030.0,
    7605049.0
])
```

La consigne demandait ensuite d'ordonner la liste ainsi obtenue avec la fonction locale fournie, nommée `ordreDecroissant()`. Plutôt que d'utiliser un tri intégré à un `DataFrame`, j'ai bien appliqué un algorithme natif à la liste elle-même. La fonction locale utilisée était :

```
#Fonction pour trier par ordre décroissant les listes (îles et populations)
def ordreDecroissant(liste):
    liste.sort(reverse = True)
    return liste
```

Et son application :

```
# Visualisation log-log rang-taille
log_surface = conversionLog(surface_sorted)
log_rangs = conversionLog(rangs)
```

Ce tri a généré une liste de surfaces fortement hiérarchisée, ce qui, sans surprise, traduit un espace extrêmement inégal en termes d'ordres de grandeur, puisque le premier rang correspond désormais à un continent, et non à une île. Cela justifie l'illisibilité du graphique rang-taille en échelle linéaire, confirmé par l'image générée dès l'étape 5. Le graphique linéaire a été produit comme suit :

```
plt.figure(figsize=(8,6))
plt.plot(log_rangs, log_surface)
plt.title("Loi rang-taille (log-log) - Îles")
plt.xlabel("log(rang)")
plt.ylabel("log(surface)")
plt.tight_layout()
plt.savefig("figures/iles_loglog.png")
plt.close()
```

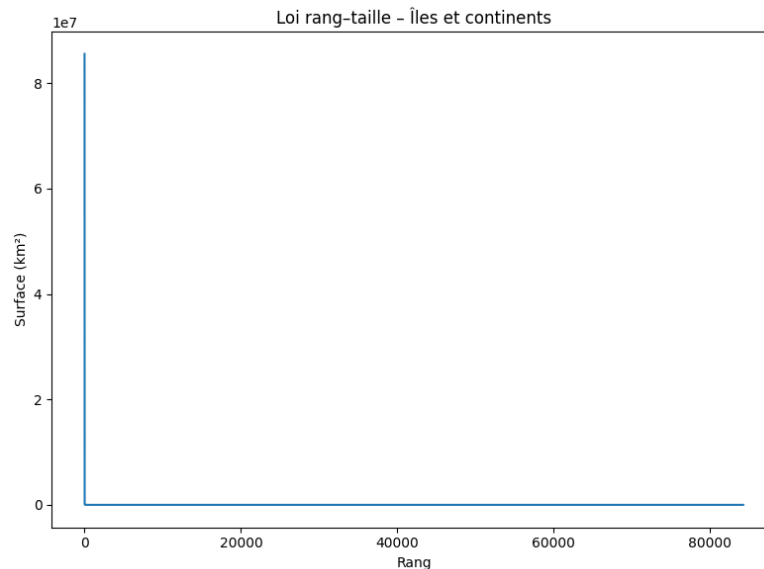


Fig. 11 : Schéma de la Loi rang-taille.

La courbe rang-taille observée montrait un premier rang écrasant (~ 85 millions km^2), puis un aplatissement brutal dès les rangs suivants, écrasés par l'écart entre les valeurs insulaires et continentales. Pour répondre à la consigne professorale visant à rendre le graphique lisible, j'ai converti les axes et surfaces en logarithme. La fonction locale `conversionLog()` fournie dans le script a alors été utilisée, prenant bien en argument une liste native. Le code de conversion était :

```
# Visualisation log-log rang-taille
log_surface = conversionLog(surface_sorted)
log_rangs = conversionLog(rangs)
```

Et la fonction indiquée :

```
#Fonction pour convertir les données en données logarithmiques
def conversionLog(liste):
    log = []
    for element in liste:
        log.append(math.log(element))
    return log
```

Le graphique log-log final a ensuite été réalisé :

```
# Visualisation log-log rang-taille
log_surface = conversionLog(surface_sorted)
```

```
log_rangs = conversionLog(rangs)

plt.figure(figsize=(8,6))
plt.plot(log_rangs, log_surface)
plt.title("Loi rang-taille (log-log) - Îles")
plt.xlabel("log(rang)")
plt.ylabel("log(surface)")
plt.tight_layout()
plt.savefig("figures/iles_loglog.png")
plt.close()
```

Ce graphique log-log a permis d'apercevoir une décroissance plus progressive, globalement linéaire, bien que marquée par un point extrême au rang 1 et une flexion en queue de distribution.

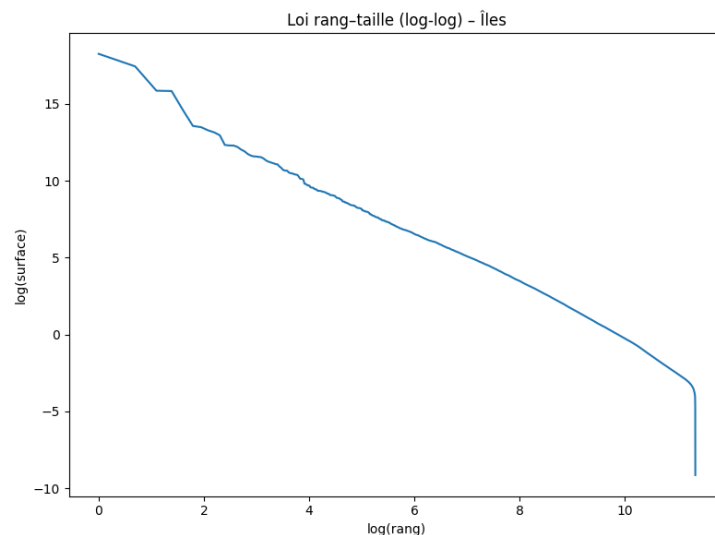


Fig. 12 : Schéma de la Loi rang-taille (log-log).

La consigne demandait ensuite s'il était possible de faire un test sur les rangs : j'y ai répondu directement sous la forme d'un commentaire intégré au script Python :

```
# Oui, il est possible de tester les rangs
# via les coefficients de Spearman et Kendall.
```

2. Fichier Le Monde (Étape 8 à 14).

Dans la seconde partie des manipulations, j'ai introduit dans le même dossier src/data le fichier global de population mondiale, nommé

Le-Monde-HS-Etats-du-monde-2007-2025.csv. Celui-ci a été ouvert de la même manière que le premier, pour rester conforme au modèle et maintenir un typage flottant si nécessaire :

```
#Partie sur les populations des États du monde
#Source. Depuis 2007, tous les ans jusque 2025, M. Forriez a relevé
l'intégralité du nombre d'habitants dans chaque États du monde proposé par
un numéro hors-série du monde intitulé États du monde. Vous avez
l'évolution de la population et de la densité par année.
df_monde = ouvrirUnFichier("./data/Le-Monde-HS-Etats-du-monde-2007-
2025.csv")
monde = pd.DataFrame(df_monde)
```

J'ai ensuite isolé les colonnes utiles pour l'analyse, à savoir : État, Pop 2007, Pop 2025, Densité 2007 et Densité 2025. Ces colonnes ont été extraites en forçant leur type au moment où elles sont ajoutées dans des listes natives, tout en respectant l'absence d'unités dans l'espace informatique. Dans mon script final, cela correspond à :

```
etat_m = monde["État"].astype(str).tolist()
pop2007_m = monde["Pop 2007"].astype(float).tolist()
pop2025_m = monde["Pop 2025"].astype(float).tolist()
dens2007_m = monde["Densité 2007"].astype(float).tolist()
dens2025_m = monde["Densité 2025"].astype(float).tolist()
```

L'étape suivante consistait à ordonner ces différentes listes de manière décroissante en utilisant la fonction locale `ordrePopulation()`, fournie dans la consigne du professeur. Cette fonction prenant en paramètres la liste à trier et la liste des États, j'ai généré quatre classements différents : pour la population en 2007, la population en 2025, la densité en 2007 et la densité en 2025.

Ces listes ont été construites en respectant la manipulation informatique directe sur les objets `list()` sans intégrer d'unités ou de structures tabulaires non natives. Pour préparer la comparaison population/densité de 2007, j'ai ensuite obtenu un tableau d'appariement entre les deux classements à l'aide de la fonction locale `classementPays()`, qui renvoie une structure de rangs croisés avec les États concordants.

Le tri Python `.sort()` a ensuite été appliqué afin d'organiser le tableau selon la hiérarchie de 2007, étape directement visible dans le terminal. Dans mon code, cette manipulation correspondait à :

```
for an in range(2007, 2026):
    c_pop = f"Pop {an}"
    c_den = f"Densité {an}"
    if c_pop in monde.columns and c_den in monde.columns:

        pop = monde[c_pop].astype(float).tolist()
        dens = monde[c_den].astype(float).tolist()

        ordre_pop = ordrePopulation(pop, etats)
        ordre_dens = ordrePopulation(dens, etats)

        if len(ordre_pop) > 0 and len(ordre_dens) > 0:
            rangs_pop = [x[0] for x in ordre_pop]
            rangs_den = [x[0] for x in ordre_dens]

            rho, tau = analyseRangs(rangs_pop, rangs_den)
            surfaces_corr.append([an, rho, tau])

        print(f"Année {an} : Spearman =", rho, ", Kendall =", tau)

# Export du bonus 2007-2025
pd.DataFrame(surfaces_corr, columns=["Année", "Spearman", "Kendall"]) \
    .to_csv("figures/corr_rangs_2007_2025.csv", index=False)

print("Bonus rangs 2007-2025 terminé : Figures et CSV dans /figures/")

#Attention ! Il va falloir utiliser des fonctions natives de Python dans
les fonctions locales que je vous propose pour faire l'exercice. Vous devez
caster l'objet Pandas en list().
```

3. Bonus.

Pour le bonus, j'ai d'abord appliqué une analyse systématique des rangs sur les îles, en comparant le classement des surfaces avec celui des longueurs de côtes. En utilisant la fonction `analyseRangs()`, j'ai obtenu :

```
BONUS Îles - Surface vs Côte → Spearman = 0.005456802490792027 , Kendall =
0.003705331884527767
```

Ce résultat indique une quasi-absence totale de corrélation, ce qui s'explique par la nature géographique des deux variables, gouvernées par des logiques très différentes.

Pour la population mondiale, j'ai automatisé l'analyse pour toutes les années de 2007 à 2025, après avoir factorisé mon code et utilisé une boucle. L'ensemble des résultats a été exporté dans le dossier figures/ sous forme de CSV :

```
# Export du bonus 2007-2025
pd.DataFrame(surfaces_corr, columns=["Année", "Spearman", "Kendall"]) \
.to_csv("figures/corr_rangs_2007_2025.csv", index=False)
```

La sortie terminale montre une stabilité temporelle des hiérarchies mondiales.

corr_rangs_2007_2025		
Année	Spearman	Kendall
2007	1.0	1.0
2008	1.0	1.0
2009	1.0	1.0
2010	1.0	1.0
2011	1.0	1.0
2012	1.0	1.0
2013	0.9999999999999999	1.0
2014	0.9999999999999999	0.9999999999999998
2015	0.9999999999999999	1.0
2016	0.9999999999999999	1.0
2017	0.9999999999999999	1.0
2018	0.9999999999999999	1.0
2019	0.9999999999999999	1.0
2020	0.9999999999999999	1.0
2021	0.9999999999999999	1.0
2022	0.9999999999999999	1.0
2023	0.9999999999999999	1.0
2024	0.9999999999999999	1.0
2025	0.9999999999999999	1.0